Gestione dei processi delle pratiche universitarie: UniGEST

Andrea Ciervo, Mathias Bartolacelli, Federico Colli

Giugno 2025

Contents

1 Introduzione **2**

2 Analisi dei requisiti **3**
2.1 Utenti . . . 3
2.2 Requisiti funzionali . . . 3
2.3 Requisiti non funzionali . . . 4
2.3.1 Sicurezza . . . 4
2.3.2 Portabilità e Scalabilità . . . 4
2.3.3 Affidabilità . . . 4
2.4 Pagine necessarie . . . 4
2.5 Requisiti di sistema . . . 5
2.5.1 Database . . . 5
2.5.2 Backend . . . 5
2.5.3 Frontend . . . 5
2.5.4 Containerizzazione . . . 5
2.5.5 Gestione del file system . . . 6

3 Metodologia di sviluppo **6**
3.1 Metodologia scelta . . . 6
3.2 Membri del team . . . 6
3.3 Fasi di sviluppo . . . 6
3.4 Diagramma di Gantt . . . 7
3.5 Versionamento . . . 7

4 Design pattern **7**
4.1 Model-View-Presenter (MVP) . . . 7
4.2 Front Controller . . . 8
4.3 Strategy . . . 8

5 Database **8**
5.1 Struttura del database . . . 8
5.2 Codice del Database . . . 9

6 Diagrammi UML **12**
6.1 Diagramma delle classi . . . 12
6.2 Casi d'uso . . . 13
6.2.1 Glossario di progetto . . . 13
6.2.2 Attori coinvolti . . . 13
6.2.3 Elenco dei casi d'uso . . . 13
6.2.4 Diagramma dei casi d'uso . . . 16
6.2.5 Matrice di tracciabilità . . . 17
6.3 Diagrammi di stato . . . 17
6.3.1 Pratica . . . 17
6.3.2 Passaggio . . . 18

6.3.3 Utente . . . 19
6.3.4 Documenti . . . 20
6.4 Diagrammi di sequenza . . . 21
6.4.1 Login . . . 21
6.4.2 Creazione delle pratiche . . . 22
6.4.3 Caricamento file . . . 23
6.4.4 Assegnamento degli amministrativi . . . 24
6.4.5 Assegnamento degli amministrativi . . . 25
6.4.6 Eliminazione della pratica . . . 26
6.4.7 Terminazione forzata delle pratiche . . . 27
6.5 Diagrammi delle attività . . . 28
6.5.1 Login . . . 28
6.5.2 Visualizzazione passaggio . . . 28
6.5.3 Apertura di una pratica . . . 28
6.5.4 Assegnazione di un passaggio . . . 29
6.5.5 Caricamento dei documenti . . . 29
6.5.6 Terminazione forzata della pratica . . . 29
6.6 Diagramma delle partizioni . . . 30
6.7 Diagrammi delle comunicazioni . . . 31

7 Diagramma dei componenti **38**

8 Diagramma di distribuzione **39**

9 Dettagli dell'implementazione **39**
9.1 Frontend . . . 39
9.1.1 Librerie e strumenti . . . 39
9.1.2 Struttura file e cartelle frontend . . . 39
9.2 Backend . . . 40
9.2.1 Librerie e strumenti . . . 40
9.2.2 Struttura file e cartelle backend . . . 41
9.2.3 Autenticazione . . . 42
9.2.4 Caricamento documenti . . . 42
9.3 COMUNICAZIONE API . . . 42

10 Testing **44**
10.1 PHPUnit . . . 44

11 Avvio **45**

1 Introduzione

L'obiettivo di questo progetto è quello di sviluppare un sistema per la descrizione e gestione dei processi aziendali, in particolare quelli universitari.
Tali processi riguardano soprattutto la gestione di pratiche amministrative. Ci sono vari tipi di attori, tra cui:

- Attori che possono richiedere l'inizio di una pratica (corpo docenti e ricercatori);
- Unità operative dedicate a specifiche mansioni;
- Amministrativi afferenti ad unità operative;
- Utenti jolly che possono intervenire in ogni installe nella gestione della pratica.

Una pratica viene quindi aperta da una specifica persona e inizia un iter di passaggi amministrativi fino al suo compimento. Ogni passaggio è affidato ad una determinata unità operativa che una volta ricevuta la pratica la assegna ad una o più amministrativi di tale unità. Ogni passaggio specifica i documenti che l'incaricato del passaggio precedente, o qualunque altro attore, deve fornire e per ogni passaggio è anche descritta la lista dei documenti prodotti.

2 Analisi dei requisiti

2.1 Utenti

Gli attori da considerare nel progetto sono suddivisi in:

- Professori/Docenti
- Ricercatori
- Amministrativi
- Direttore
- Unità Operative
- Utenti Jolly

La gestione degli attori verrà effettuata tramite una specializzazione di una classe base detta Utente dalla quale deriveranno gli attori Professori/Docenti, Ricercatori e Amministrativi. Gli amministrativi a loro volta saranno collegati a un'unità operativa della quale sarà indicato un direttore. Per quanto riguarda gli utenti Jolly, questi non saranno una specializzazione ma saranno definiti tramite un attributo booleano all'interno del database (di base indicato a false) il quale sarà modificabile solo dal database.
Siccome questa applicazione verrà integrata nel sistema informatico dell'ateneo, non è prevista la registrazione di nuovi utenti.

2.2 Requisiti funzionali

- **Gestione delle pratiche**
  - Le pratiche saranno aperte da Professori/Docenti o Ricercatori i quali, dopo aver effettuato l'accesso, tramite l'apposito bottone della creazione pratiche ne potranno realizzare una.
  - Durante la creazione sarà necessario scrivere una piccola descrizione della richiesta, caricare un primo documento d'avvio e la tipologia della richiesta (la quale sarà presa da un elenco realizzato a livello di database con annesso il numero di passaggi per il completamento e i documenti da realizzare per passaggio).
  - Al completamento della richiesta questa sarà presa in carico dal direttore dell'unità operativa del primo passaggio il quale assegnerà la pratica a uno o più amministrativi.
  - Al termine del passaggio, che verrà effettuato in automatico, verrà aperta la pagina della gestione agli amministrativi assegnati al passaggio successivo della pratica. Al termine dell'ultimo passaggio lo stato della pratica viene modificato per definire la sua terminazione.
    Il Jolly può anche forzare la chiusura di una pratica, indicandone il motivo.

- **Gestione del passaggio**

  Dopo che un passaggio viene assegnato a uno o più amministrativi questi produrranno i documenti necessari al singolo passaggio, i quali saranno salvati sull'area di memoria del server in cartelle indicate con l'id della pratica e il numero del passaggio, il quale sarà da -1(presenta i documenti caricati all'avvio dall'utente) a n-1, così da essere reperibili ai passaggi

che le richiedono. Potranno essere visibili agli amministrativi dei passaggi successivi così da rispondere alla richiesta "Ogni passaggio specifica i documenti che l'incaricato del passaggio precedente, o qualunque altro attore, deve fornire." e potranno essere anche utilizzati per stilare automaticamente la lista dei documenti prodotti durante il passaggio. Per effettuare la carica si dovrà necessitare di un pulsante presente nella pagina di dettaglio del passaggio accessibile agli amministratori o al Jolly.

- **Caricamento documenti**
  Il caricamento dei documenti verrà effettuato dagli utenti amministrativi e Jolly. Questo sarà effettuato in seguito all'assegnazione del passaggio per l'Amministrativo, solo sul suo attuale passaggio; mentre per il Jolly potrà essere effettuato in qualsiasi momento su qualsiasi passaggio.Inoltre anche l'utente base potrà caricare dei file, i quali saranno inseriti solamente nella cartella -1 dei passaggi e saranno caricati solo al momento della creazione della pratica, all'interno del selettore file presente nel form di caricamento.

- **Gestione con jolly**
  Il Jolly avrà la possibilità di osservare tutte le pratiche e i relativi passaggi, assegnare le pratiche agli amministrativi dell'unità indicata ed eventualmente forzare la chiusura o l'eliminazione di una pratica.

2.3 Requisiti non funzionali

2.3.1 Sicurezza

La sicurezza è garantita tramite:

- privilegi basati sul ruolo (utenti, amministrativi, direttori e jolly)
- gestione token tramite JWT
- validazione utenti per effettuare modifiche
- hashing password lato server

2.3.2 Portabilità e Scalabilità

Portabilità e scalabilità sono garantite tramite la containerizzazione.

2.3.3 Affidabilità

L'affidabilità è garantita tramite la gestione dei rollback sul database per prevenire inconsistenze in caso di errori durante le operazioni di salvataggio o eliminazione dei dati. Questo approccio permetterà di annullare automaticamente eventuali modifiche parziali se un'operazione non viene completata correttamente, assicurando così l'integrità delle informazioni e la coerenza dei dati nel database.

2.4 Pagine necessarie

- Pagina di login
- Pagina con la lista delle pratiche aperte dall'utente (Per docenti e ricercatori)
- Pagina con il form di creazione delle pratiche (Per docenti e ricercatori)
- Pagina di assegnazione delle pratiche (Direttore dell'u. operativa)
- Pagina di visualizzazione dei passaggi assegnati (Per amministrativi)
- Pagina per caricamento File (Per amministrativi e Jolly)

Nota: gli utenti Jolly possono avere accesso a tutte queste funzionalità e, inoltre, possono eliminare o terminare anticipatamente le pratiche in corso

2.5 Requisiti di sistema

2.5.1 Database

Il database relazionale utilizzato è mySQL e si compone delle seguenti tabelle:

- Utente, che contiene le informazioni di accesso di tutti gli utenti, inclusi username e password cifrata
- Docente, che contiene una lista degli utenti che sono docenti
- Ricercatore, che contiene una lista degli utenti che sono ricercatori
- Amministrativo, che contiene una lista degli utenti che sono amministrativi, l'unità a cui appartengono e un bit che indica se sono direttori della propria unità
- U. Operativa, che contiene una lista di tutte le unità operative
- Assegnazione, che contiene i passaggi associati ad ogni amministrativo
- Passaggio, che contiene una lista di informazioni relative ad ogni passaggio di ogni pratica
- Pratica, contiene una lista di tutte le pratiche, aperte e non
- Tipologia, che contiene, per ogni tipologia di pratica, informazioni quali il numero di passaggi e i passaggi

2.5.2 Backend

L'applicazione sarà sviluppata utilizzando PHP, seguendo un'architettura basata sul pattern Model-View-Presenter (MVP) e mettendo a disposizione API REST tramite il presenter per l'interazione con il frontend.
In particolare:

- Il Controller si occuperà di gestire le richieste HTTP e di indirizzarle verso i gateway appropriati,
- Il Gateway avrà il compito di interagire con il model, fornendo all'utente le funzionalità richieste in base al percorso della richiesta,
- Il Model sarà responsabile delle operazioni dirette sul database relative a una specifica entità.

2.5.3 Frontend

Il frontend verrà sviluppato utilizzando HTML, CSS e Javascript, la libreria JQuery per la gestione degli eventi tramite chiamate AJAX e il framework Bootstrap per sfruttare strutture già realizzate. In particolare, per ogni pagina verrà creato un file HTML e un file JS e le funzionalità in comune tra più pagine verranno inserite in un file a parte.

2.5.4 Containerizzazione

Il sistema dovrà essere diviso in container specializzati, ognuno dei quali sarà progettato per gestire un singolo servizio o componente specifico. Questa scelta architetturale permetterà di isolare le varie funzionalità, migliorando la sicurezza, la stabilità e la manutenzione complessiva dell'applicazione.

In particolare verranno costruiti i seguenti container:

- un web-server per il frontend, che si occuperà di gestire solamente le pagine HTML e la comunicazione con il server backend tramite richieste JSON
- un web-server per il backend, che si occuperà di gestire:

  - le richieste ricevute,
  - le interazioni con il database
  - il salvataggio file sul filesystem del backend

- un container che gestisce il PHPmyAdmin il quale permette una visualizzazione grafica del database

- un container che gestisce il database per conservare i dati al suo interno

2.5.5 Gestione del file system

Il sistema dovrà prevedere la gestione del filesystem, che consenta sia il caricamento di file da parte degli utenti sia la creazione automatica di cartelle dedicate al salvataggio dei documenti.
Ogni pratica dovrà avere una propria cartella principale, all'interno della quale saranno contenute le sottocartelle corrispondenti ai vari passaggi della pratica.

3 Metodologia di sviluppo

3.1 Metodologia scelta

Per questo progetto sarà adottata una metodologia di sviluppo basata sull'Extreme Programming che permetterà, tramite lo sviluppo e la consegna di piccoli incrementi di funzionalità ad ogni step, di adattarci velocemente ai cambiamenti dei requisiti, garantendo al tempo stesso una struttura di lavoro chiara e ben organizzata.
Si è adottata l'**EP** poiché la realizzazione del progetto è stata attuata tramite il pair programming, così da avere uno scambio immediato di idee e la partecipazione continua di tutti i membri del gruppo allo sviluppo del codice in quantità più o meno elevata.

3.2 Membri del team

Il team di sviluppo sarà composto dai seguenti componenti:

- un(1) Project Manager, che si occuperà di gestire il team e prendere decisioni sulla progettazione e lo sviluppo e di gestire il versionamento del codice sorgente;

- un(1) Backend developer, che si occuperà, insieme al project manager, della scrittura del codice relativo al backend e alla gestione dei database;

- un(1) Frontend developer, che si occuperà della scrittura del codice relativo al Frontend dell'applicazione e la gestione delle richieste HTTP.

3.3 Fasi di sviluppo

Il lavoro è stato diviso in parti principali, ognuna composta da molti step, così da favorire una gestione incrementale e continua delle funzionalità. Le fasi sono:

- progettazione, che comprende la decisione sulla struttura del database, dei docker, delle api e delle pagine web

- scrittura del docker compose e creazione del codice di inizializzazione del database

- scrittura codice del login con integrazione delle parti, sia lato frontend che backend

- scrittura codice per l'utente base (Docente o Ricercatore) sia lato frontend che backend

- scrittura codice per Amministrativi e Direttori

- scrittura codice per Jolly, che comprende l'inclusione di tutte le funzionalità e l'aggiunta di altre

Al termine di ogni fase verrà eseguita una fase di testing del relativo modulo, che permette di controllare se tutte le funzionalità create operino correttamente. Inoltre verrà effettuata una revisione degli obiettivi al fine di verificare che tutto sia stato svolto correttamente e, in caso contrario, operare per la correzione.

3.4 Diagramma di Gantt

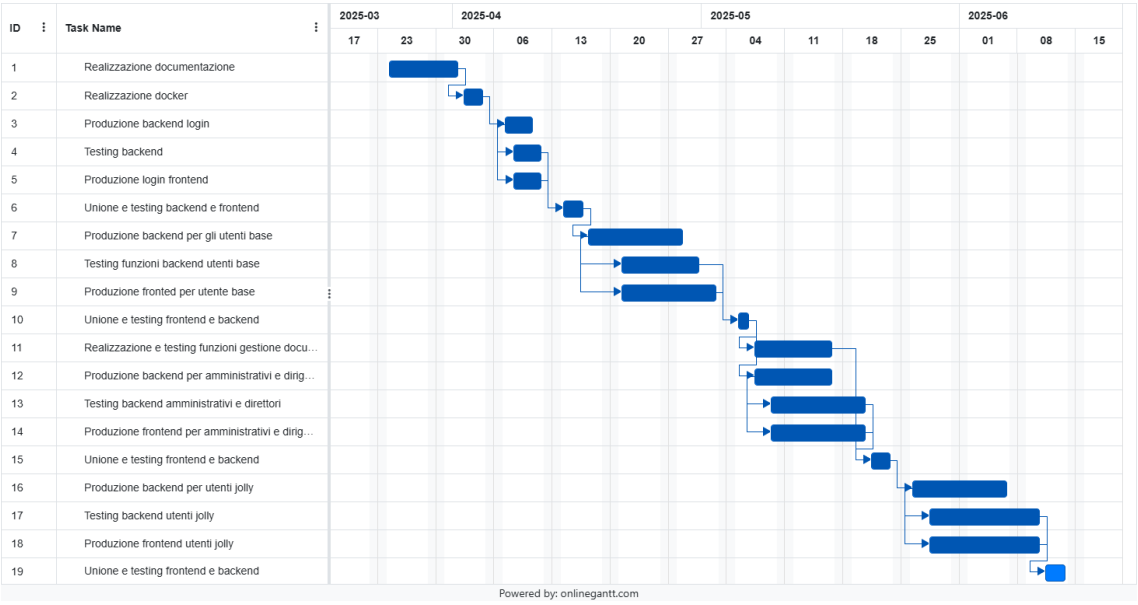

Figure 1: Diagramma di Gantt

3.5 Versionamento

Per gestire il versionamento del codice è stato usato GitHub, che permette di lavorare in modo organizzato attraverso l'uso dei branch, così da evitare conflitti e mantenere stabile la versione principale del progetto.

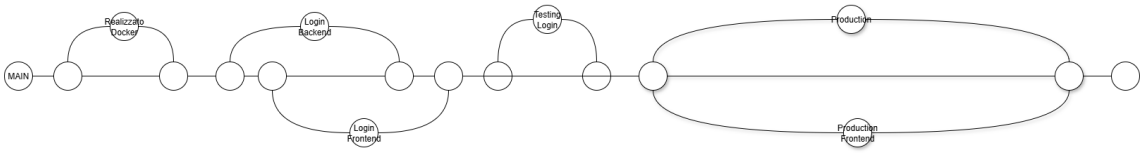

Figure 2: Branching del progetto

4 Design pattern

4.1 Model-View-Presenter (MVP)

Il design patter MVP è stato utilizzato nello sviluppo dell'applicazione, in particolare:

- Model: gestisce l'accesso diretto al database, occupandosi delle operazioni di recupero, inserimento e modifica dei dati;
- View: rappresentato dalle pagine HTML e javascript del servizio;
- Presenter: composto dai Gateway, responsabili della logica di business specifica per ogni funzionalità.

4.2 Front Controller

Il pattern Front Controller prevede un unico punto di ingresso per la gestione centralizzata delle richieste, favorendo un'organizzazione più coerente del flusso applicativo.
Nel progetto, il file index.php svolge questo ruolo nel backend: intercetta tutte le richieste in ingresso e delega il loro smistamento a Controller.php, che "spacchetta" l'url ricevuto e smista le richieste ai Gateway responsabili delle singole funzionalità. Questa struttura facilita la gestione del routing, centralizza la logica di controllo e rende l'applicazione più manutenibile ed estensibile.
Allo stesso modo, nel frontend, index.html funge da punto di accesso unico e poi, tramite le informazioni sul ruolo e l'identità di chi fa l'accesso (ricevuto dal backend), carica la pagina e i dati corrispondenti.

4.3 Strategy

Il pattern Strategy, seppur in minima parte, è stato implementato per la gestione dei getPratiche per le varie tipologie di utente.
E' stato realizzato all'interno della classe Pratica.php nella quale sono state realizzate funzioni differenti con lo stesso scopo, ovvero la restituzione dei dati delle pratiche create; queste sono state poi incapsulate all'interno delle classi di Utente, Amministrativo, Direttore e Jolly all'interno di funzioni apposite.

5 Database

5.1 Struttura del database

- **Utente: Id**, Nome, Mail, Pwd, DataCreazione, Jolly
- **Docente: IdDocente***, Corso
- **Ricercatore: IdRicercatore***, Ricerca
- **Amministrativo: IdAmministrativo***, IdUnità*, Direttore
- **UOperativa: IdUnita**, Nome
- **Assegnazione: IdAmministrativo*, IdPassaggio***
- **Passaggio: IdPassaggio**, NPassaggio, IdPratica*, IdUnita*, ListDocUscita, ListDocRichiesti, Terminato
- **Pratica: IdPratica**, IdDocente*, Tipologia*, Descrizione, DataCreazione, Codice
- **Tipologia: Tipologia**, NPassaggi, Passaggi, Inizio

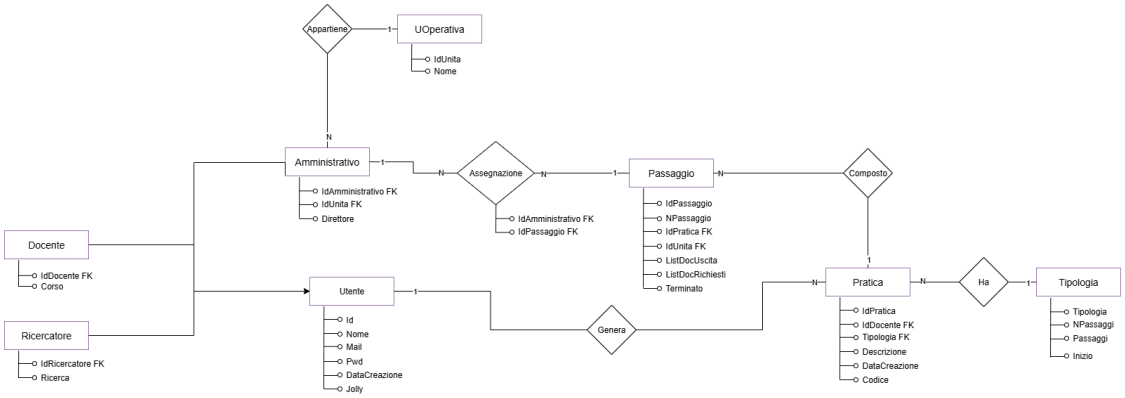

Figure 3: Schema ER

5.2 Codice del Database

```
CREATE DATABASE IF NOT EXISTS unigest;
use unigest;

CREATE TABLE IF NOT EXISTS Utente(
    Id int(5) NOT NULL AUTO_INCREMENT PRIMARY KEY,
    Nome varchar(255) not null,
    Mail varchar(255) not null,
    Pwd varchar(255) not null,
    DataCreazione datetime NOT NULL DEFAULT current_timestamp(),
    #tipe?
    Jolly boolean DEFAULT 0,
    UNIQUE(Mail, DataCreazione)
);

CREATE TABLE IF NOT EXISTS Docente(
    IdDocente int(5) NOT NULL PRIMARY KEY,
    Corso varchar(255) NOT NULL,
    FOREIGN KEY (IdDocente) REFERENCES Utente(Id)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Ricercatore(
    IdRicercatore int(5) NOT NULL PRIMARY KEY,
    Ricerca varchar(255) NOT NULL,
    FOREIGN KEY (IdRicercatore) REFERENCES Utente(Id)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS UOperativa(
    IdUnita int(5) NOT NULL AUTO_INCREMENT PRIMARY KEY,
    Nome Text(255) NOT NULL
);

CREATE TABLE IF NOT EXISTS Amministrativo(
    IdAmministrativo int(5) NOT NULL PRIMARY KEY,
    IdUnita int(5) NOT NULL,
    Direttore boolean DEFAULT 0,
    FOREIGN KEY (IdAmministrativo) REFERENCES Utente(Id)
    ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (idUnita) REFERENCES UOperativa(IdUnita)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Tipologia(
    Tipologia VARCHAR(255) NOT NULL PRIMARY KEY,
    NPassaggi int(2) NOT NULL,
    Passaggi JSON
    Inizio VARCHAR(255) NOT NULL
);

CREATE TABLE IF NOT EXISTS Pratica(
    IdPratica int(5) NOT NULL AUTO_INCREMENT PRIMARY KEY,
    IdDocente int(5) NOT NULL,
    Descrizione Text(65535) NOT NULL,
```

```
    Tipologia VARCHAR(255) NOT NULL,
    DataCreazione datetime NOT NULL DEFAULT current_timestamp(),
    Codice VARCHAR(255) DEFAULT 0,
    FOREIGN KEY (IdDocente) REFERENCES Utente(Id)
    ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (Tipologia) REFERENCES Tipologia(Tipologia)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Passaggio(
    IdPassaggio int(5) AUTO_INCREMENT PRIMARY KEY,
    NPassaggio int(3) DEFAULT 1,
    IdPratica int(5),
    IdUnita int(5),
    ListDocUscita Text(65535) NOT NULL,
    ListDocRichiesti Text(65535) NOT NULL,
    Terminato boolean DEFAULT 0,
    FOREIGN KEY (IdPratica) REFERENCES Pratica(IdPratica)
    ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (IdUnita) REFERENCES UOperativa(IdUnita)
    ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE IF NOT EXISTS Assegnazione(
    IdAmministrativo int(5) NOT NULL,
    IdPassaggio int(5) NOT NULL,
    PRIMARY KEY (IdAmministrativo, IdPassaggio),
    FOREIGN KEY (IdAmministrativo) REFERENCES Amministrativo(IdAmministrativo)
    ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (IdPassaggio) REFERENCES Passaggio(IdPassaggio)
    ON DELETE CASCADE ON UPDATE CASCADE
);
```

6 Diagrammi UML

6.1 Diagramma delle classi

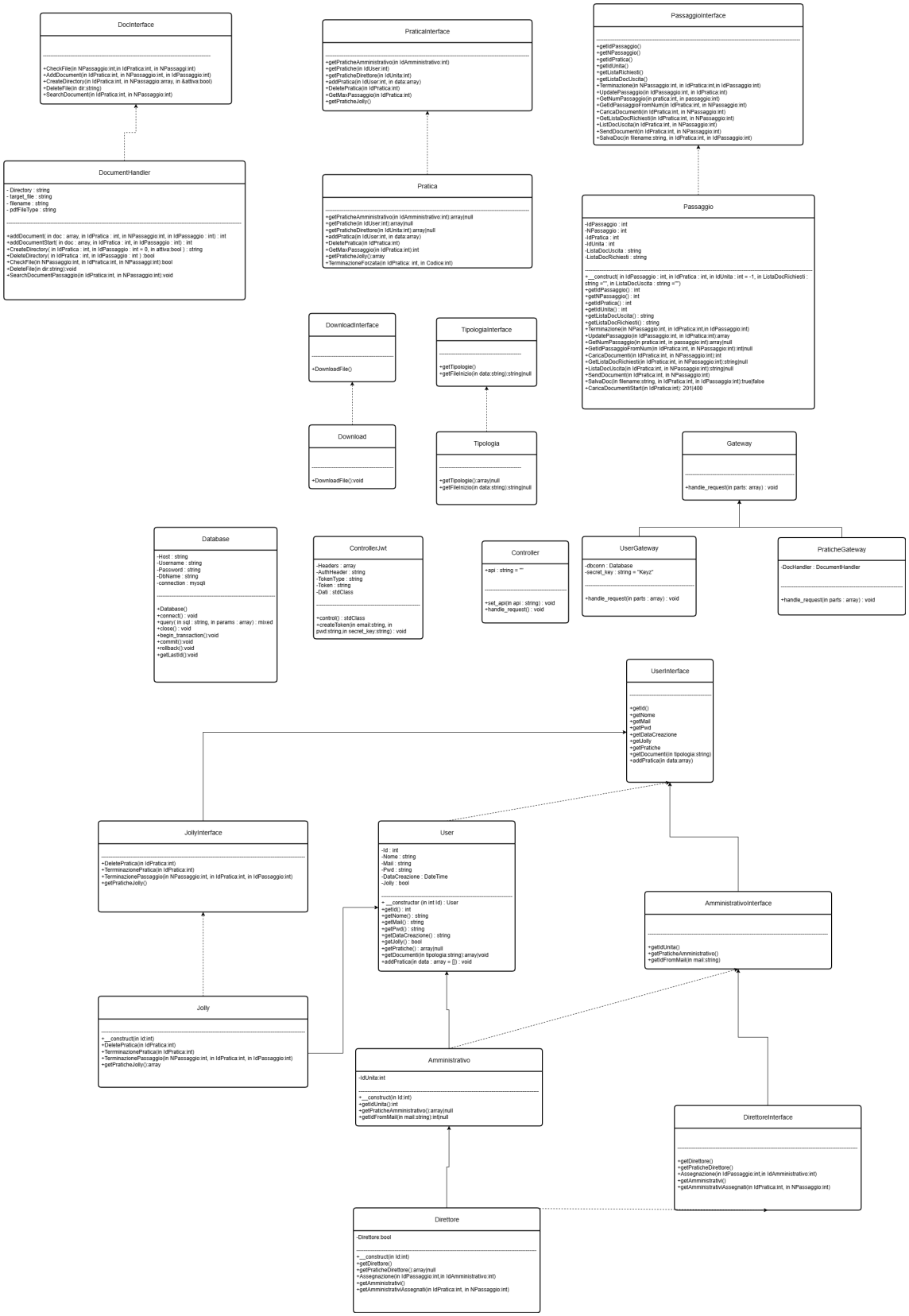

Figure 4: Diagramma delle classi

6.2 Casi d'uso

6.2.1 Glossario di progetto

- **JWT**: JSON Web Token, è un token in formato compatto usato per scambiare informazioni in modo sicuro tra client e server. Contiene un header, un payload con i dati (durata del token, email, tempo di creazione, tipologia di utente e un valore che indica se è jolly) e una firma per garantirne l'integrità;
- **Tipologia**: cosa può fare un utente all'interno della piattaforma (suddiviso in docente/ricercatore, amministrativo, direttore e jolly);
- **Utente**: una qualsiasi persona che possiede delle credenziali per accedere alla piattaforma;
- **Credenziali**: un insieme di email e password collegate ad un utente;
- **Dashboard**: la pagina principale, che contiene una lista con tutte le pratiche o passaggi relative ad un determinato utente.

6.2.2 Attori coinvolti

- Docenti e Ricercatori: hanno la necessità di utilizzare il sistema per aprire delle pratiche universitarie;
- Amministrativi: gestiscono le pratiche aperte dai professori relative alla propria unità operativa;
- Direttori: assegnano gli amministrativi della propria unità operativa alle pratiche ancora da gestire;
- Jolly: sono utenti che possono fare tutto quello che fanno gli altri utenti e, in più, possono eliminare delle pratiche ancora in corso o terminarle anticipatamente.

6.2.3 Elenco dei casi d'uso

1. Login
   - Descrizione: gli utenti devono accedere alla piattaforma utilizzando credenziali private;
   - Attori: Tutti
   - Precondizioni: Gli utenti hanno delle credenziali valide
   - Flusso principale:
     - L'utente apre la pagina di login;
     - L'utente inserisce email e password;
     - Il sistema controlla la correttezza delle credenziali
     - Se le credenziali sono corretti viene generato un token JWT associato all'utente e alla sessione in corso, che viene salvato nel local storage e verrà utilizzato per l'autenticazione delle successive richieste.
   - Flusso alternativo:
     - Se le credenziali sono errate viene generato un errore.
   - Postcondizioni: L'utente ha accesso alla piattaforma e può effettuare delle richieste.
2. Visualizzazione delle pratiche in corso
   - Descrizione: L'utente registrato può visualizzare le pratiche relative a:
     - Sé stesso se è un docente;
     - Tutte le pratiche che gli sono state assegnate se è un amministrativo;
     - Tutte le pratiche relative alla sua unità operativa se è un direttore;
     - Tutte le pratiche in corso se è un jolly.

- Attori: Tutti.
- Precondizioni: L'utente ha un JWT valido.
- Flusso principale:
  - L'utente accede alla piattaforma.
- Flusso alternativo:
  - L'utente, se non ha un JWT valido, viene reindirizzato alla pagina di login.
- Postcondizioni: L'utente ha visualizzato le pratiche e può gestirle come preferisce.

3. Creazione di una nuova pratica
   - Descrizione: L'utente crea una nuova pratica inserendone la tipologia e una descrizione.
   - Attori: Docenti, Ricercatori e Jolly.
   - Precondizioni: L'utente ha un JWT valido.
   - Flusso principale:
     - L'utente clicca sulla relativa icona nella navbar laterale;
     - L'utente inserisce tipologia e descrizione della pratica;
     - L'utente preme il pulsante "Invia".
   - Flusso alternativo:
     - Se la descrizione è vuota viene ritornato un errore.
   - Postcondizioni: L'utente avrà creato una nuova pratica, che dovrà essere gestita da un amministrativo.
4. Caricamento file
   - Descrizione: Uno o più file relativi ad una pratica vengono caricati sulla piattaforma.
   - Attori: Amministrativi e Jolly
   - Precondizioni: L'utente ha un JWT valido. Se è un amministrativo, la pratica deve essergli stata assegnata.
   - Flusso principale:
     - L'utente clicca sul pulsante "Visualizza";
     - L'utente si trova nella schermata di gestione della pratica e clicca sulla sezione relativa al caricamento dei file;
     - L'utente seleziona dalla finestra che si è aperta tutti i file che vuole caricare;
     - L'utente clicca il pulsante "OK".
     - Se tra i file caricati ci sono anche i file necessari per questo passaggio, il passaggio viene considerato completato.
     - Si torna alla dashboard.
   - Flusso alternativo:
     - Se l'utente non ha caricato i file nominati correttamente, il passaggio non termina.
   - Postcondizioni: Il passaggio è stato completato correttamente e, se tutti i passaggi della pratica sono stati completati correttamente, la pratica viene considerata terminata.
5. Assegnazione della pratica
   - Descrizione: Uno o più amministrativi vengono assegnati alla gestione di una pratica.
   - Attori: Direttore e Jolly.
   - Precondizioni: L'utente deve avere un JWT valido. Se l'utente è un direttore, la pratica da assegnare deve essere relativa all'unità operativa di cui è direttore.
   - Flusso principale:
     - L'utente clicca sul tasto "Visualizza";

    - L'utente, ora nella pagina di gestione della pratica, clicca tutti gli amministrativi che vuole assegnare a questa pratica nella relativa sezione;
    - L'utente clicca il tasto "Conferma".
  - Flusso alternativo: Se non è stato selezionato nessun amministrativo viene ritornato un errore.
  - Postcondizioni: Uno o più amministrativi vengono assegnati alla gestione di una pratica.

6. Elimina pratica
  - Descrizione: Una pratica ancora in corso viene eliminata.
  - Attori: Jolly.
  - Precondizioni: L'utente ha un JWT valido. La pratica è ancora in corso.
  - Flusso principale:
    - L'utente clicca sul pulsante "Visualizza" della pratica da gestire.
    - L'utente clicca il pulsante "Elimina pratica" in alto a destra.
    - Nel riquadro di conferma, l'utente clicca il pulsante "Conferma".
  - Flusso alternativo: Se l'utente clicca il pulsante "Annulla", la pratica non viene eliminata.
  - Postcondizioni: La pratica e tutti i suoi passaggi vengono cancellati dal database.

7. Termina pratica
  - Descrizione: Una pratica viene terminata in modo forzato.
  - Attori: Jolly.
  - Precondizioni: L'utente ha un JWT valido. La pratica è in corso(non ancora terminata).
  - Flusso principale:
    - L'utente clicca il pulsante "Visualizza" della pratica da gestire.
    - L'utente clicca il pulsante "Termina pratica".
    - L'utente inserisce il codice della causa di terminazione
    - L'utente clicca il pulsante "Conferma".
    - Il sistema termina questa pratica anche se non sono stati caricati tutti i documenti richiesti.
  - Flusso alternativo: Se l'utente clicca il pulsante "Annulla", la pratica non viene terminata.
  - Postcondizioni: La pratica viene terminata e il sistema aggiorna il database di conseguenza.

8. Logout
  - Descrizione: L'utente chiude la sessione attiva.
  - Attori: Tutti.
  - Precondizioni: L'utente ha un JWT valido.
  - Flusso principale:
    - L'utente clicca l'icona con la porta.
    - Il JWT viene cancellato dalla memoria del browser.
    - L'utente viene spostato alla pagina di login.
  - Flusso alternativo: No.
  - Postcondizioni: L'utente non può più accedere alla piattaforma se non effettua nuovamente il login.

6.2.4 Diagramma dei casi d'uso

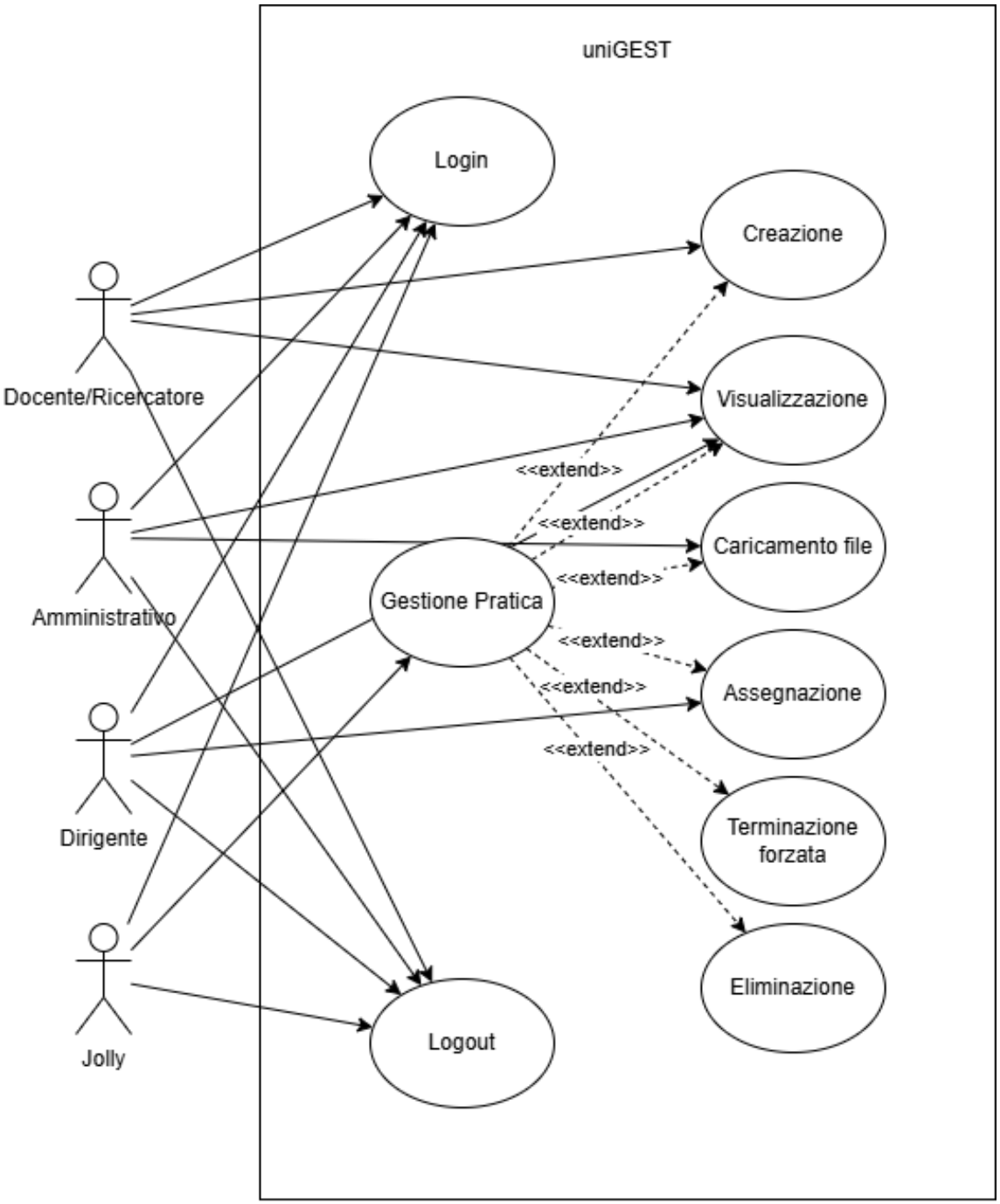

Figure 5: Diagramma dei casi d'uso

6.2.5 Matrice di tracciabilità

	1	2	3	4	5	6	7	8
Login	X							
Creazione			X					
Visualizzazione		X						
Caricamento file				X				
Assegnazione amministrativi					X			
Terminazione forzata							X	
Eliminazione						X		
Logout								X

Figure 6: Matrice di tracciabilità

6.3 Diagrammi di stato

6.3.1 Pratica

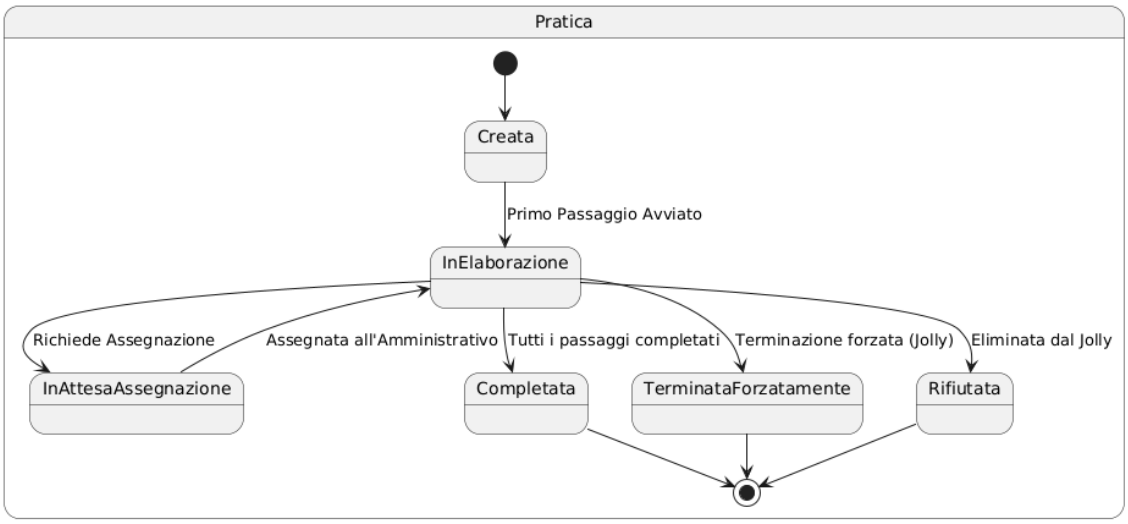

Figure 7: Diagramma degli stati di una pratica

6.3.2 Passaggio

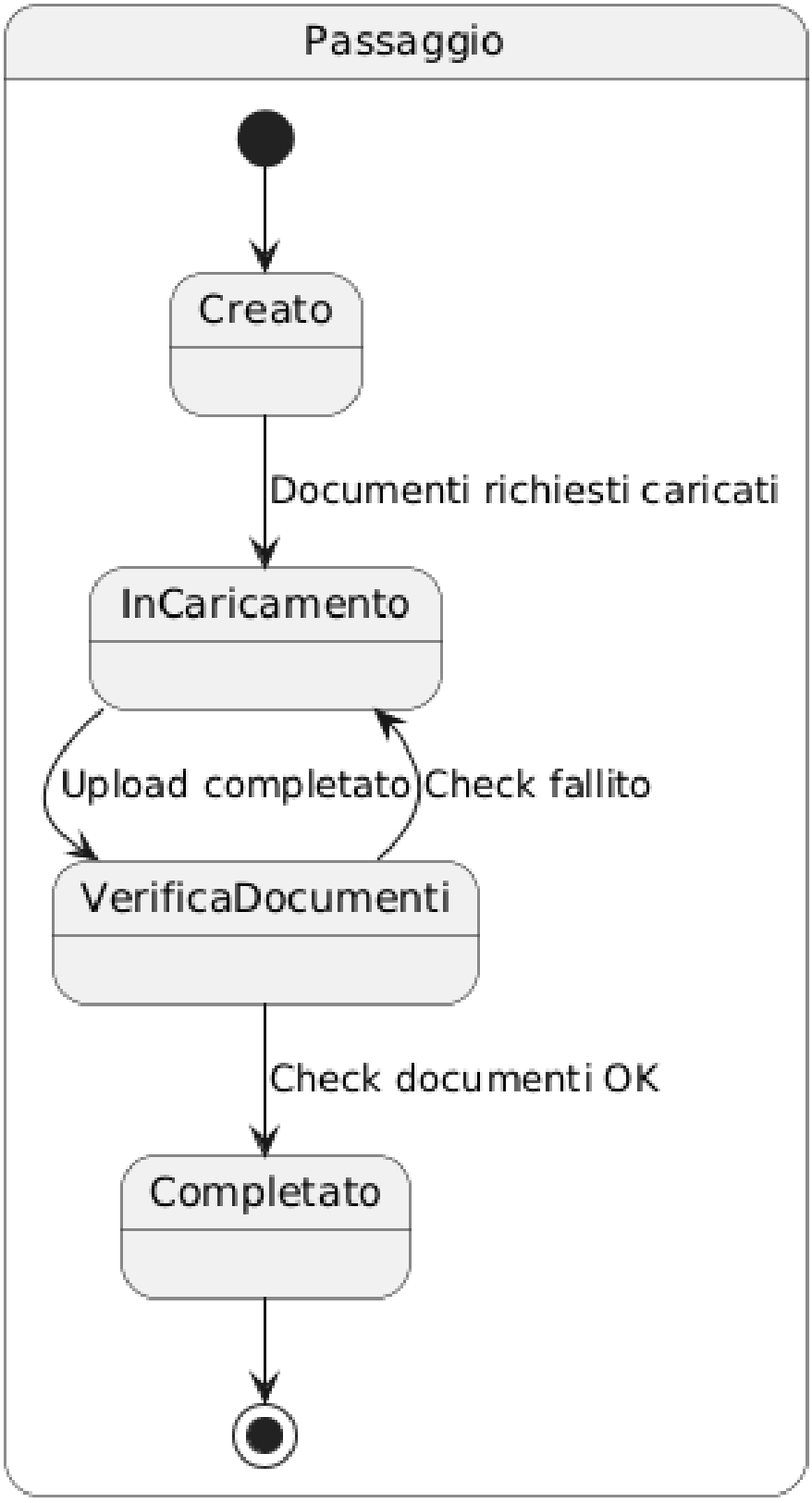

Figure 8: Diagramma degli stati di un passaggio

6.3.3 Utente

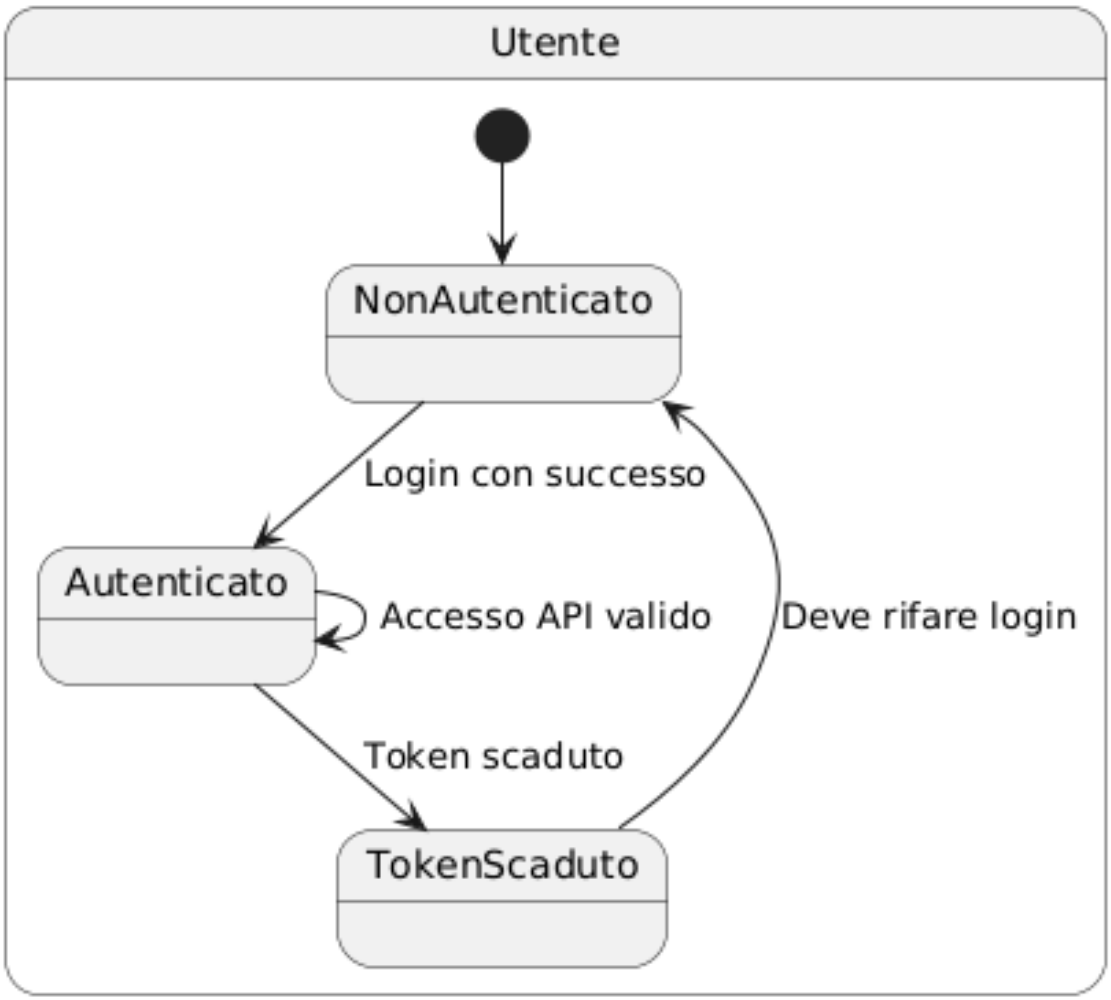

Figure 9: Diagramma degli stati di un utente

6.3.4 Documenti

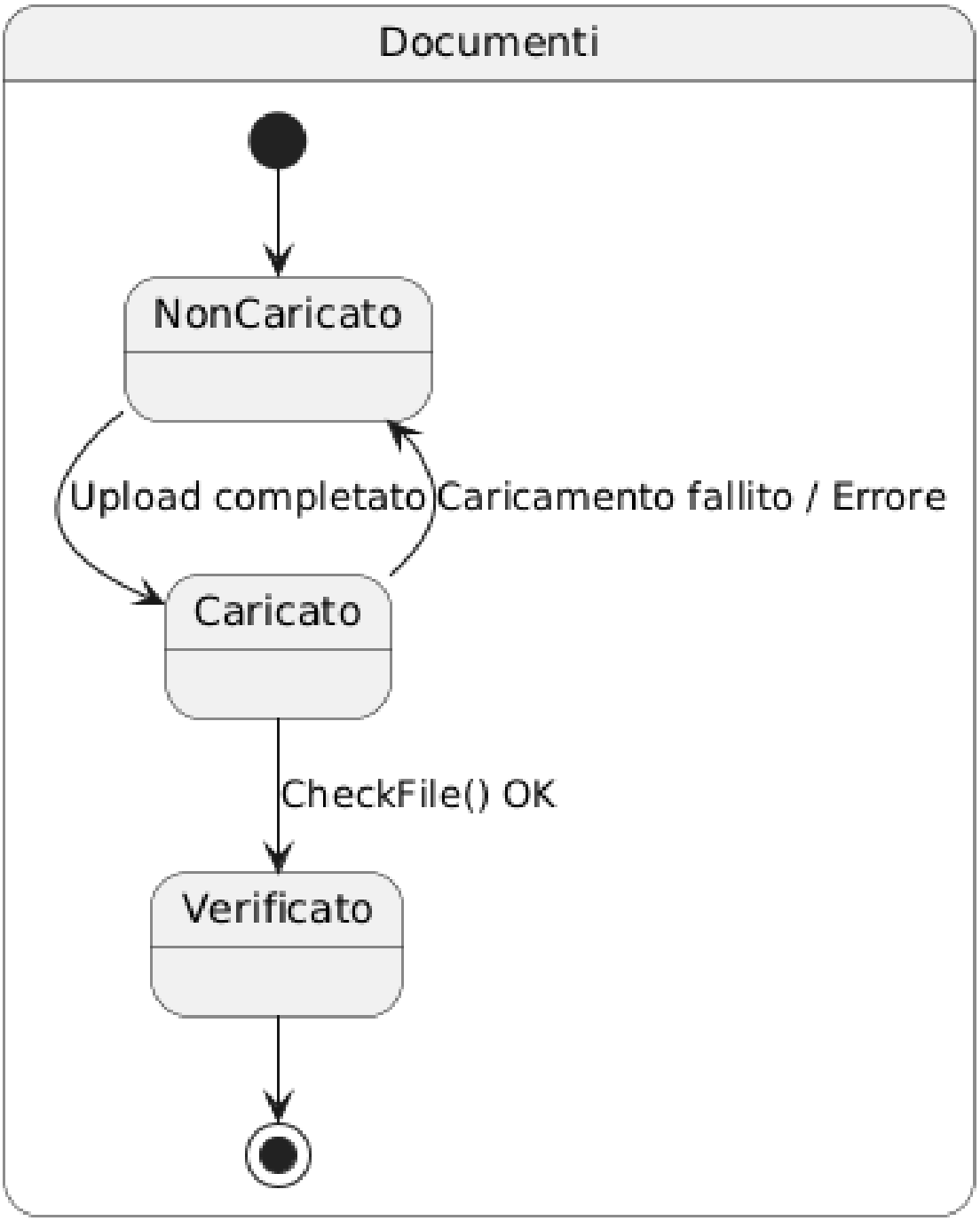

Figure 10: Diagramma degli stati di un documento

6.4 Diagrammi di sequenza

6.4.1 Login

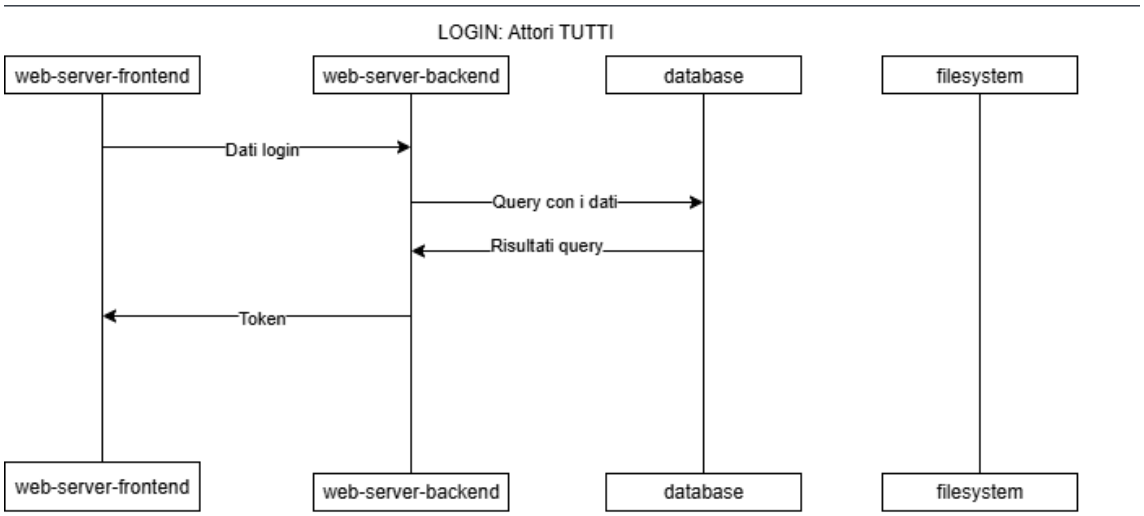

Figure 11: Diagramma di login

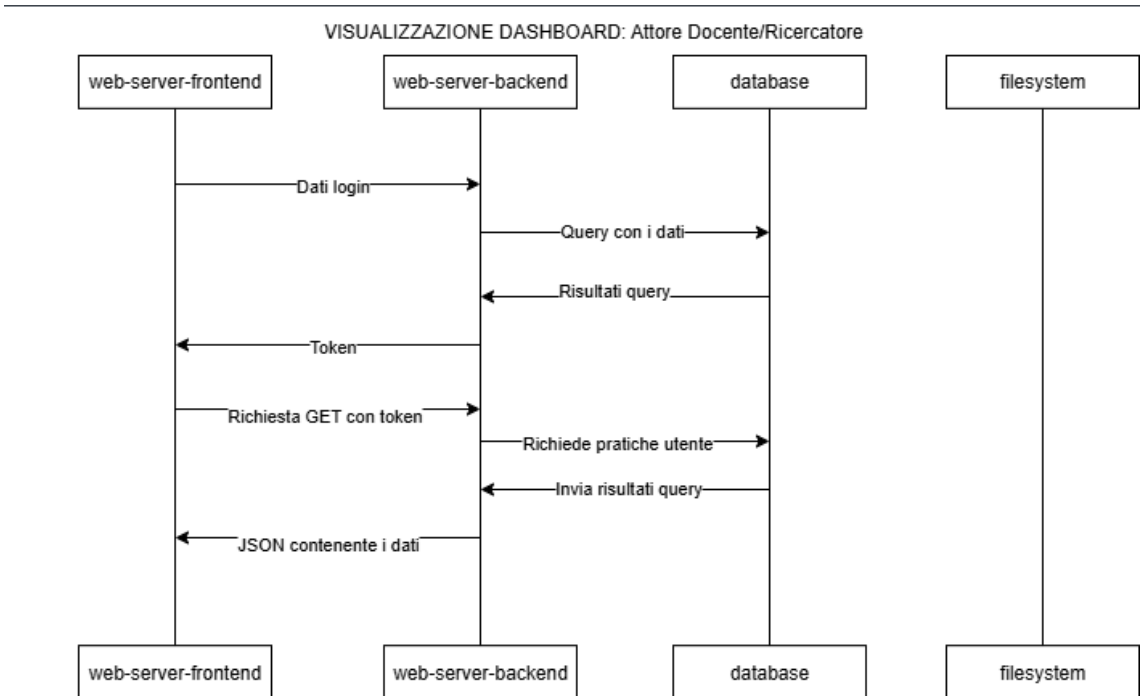

Figure 12: Diagramma della dashboard

6.4.2 Creazione delle pratiche

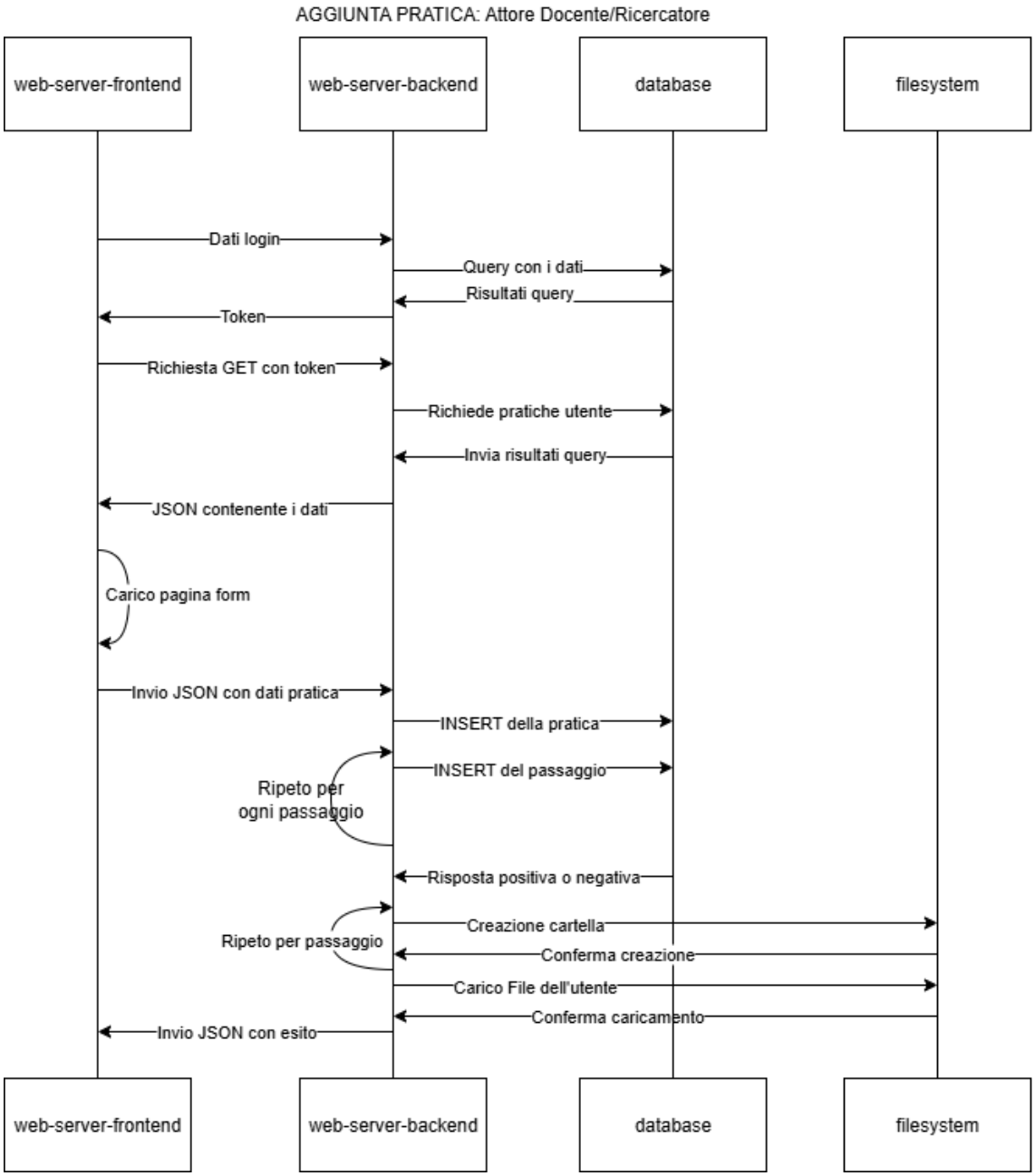

Figure 13: Diagramma di creazione delle pratiche

6.4.3 Caricamento file

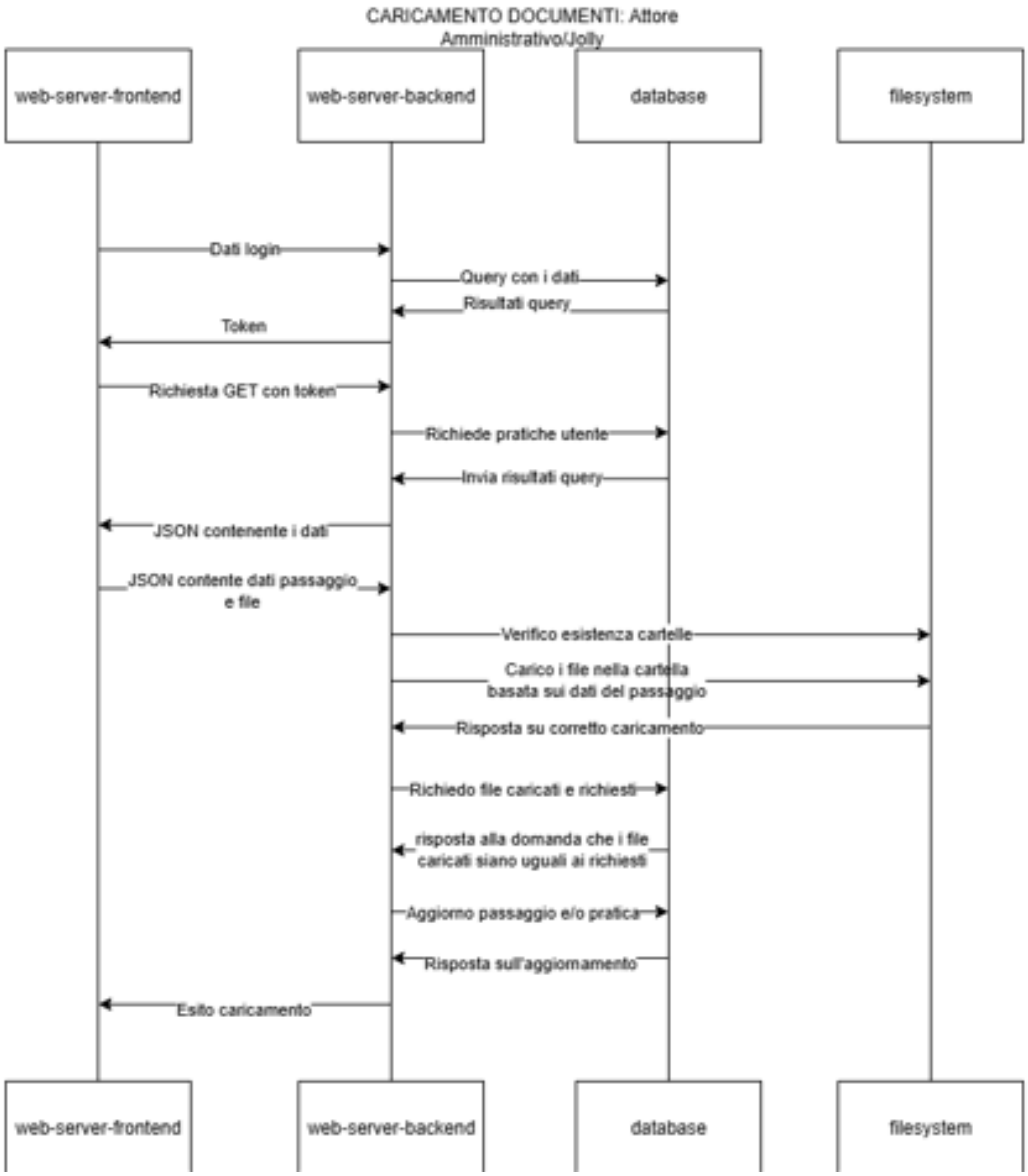

Figure 14: Diagramma del caricamento dei file

6.4.4 Assegnamento degli amministrativi

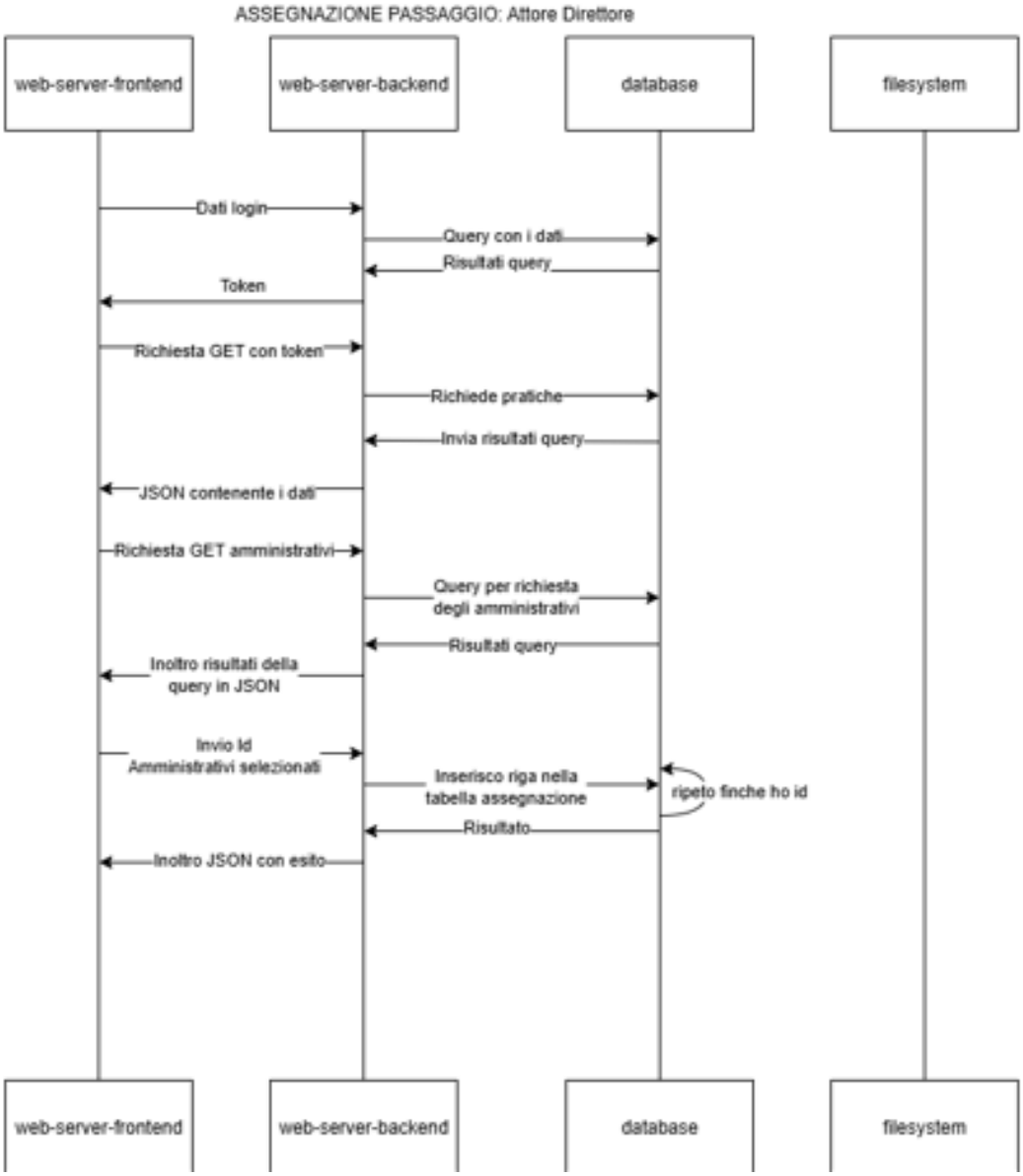

Figure 15: Diagramma dell'assegnamento degli amministrativi con direttore

6.4.5 Assegnamento degli amministrativi

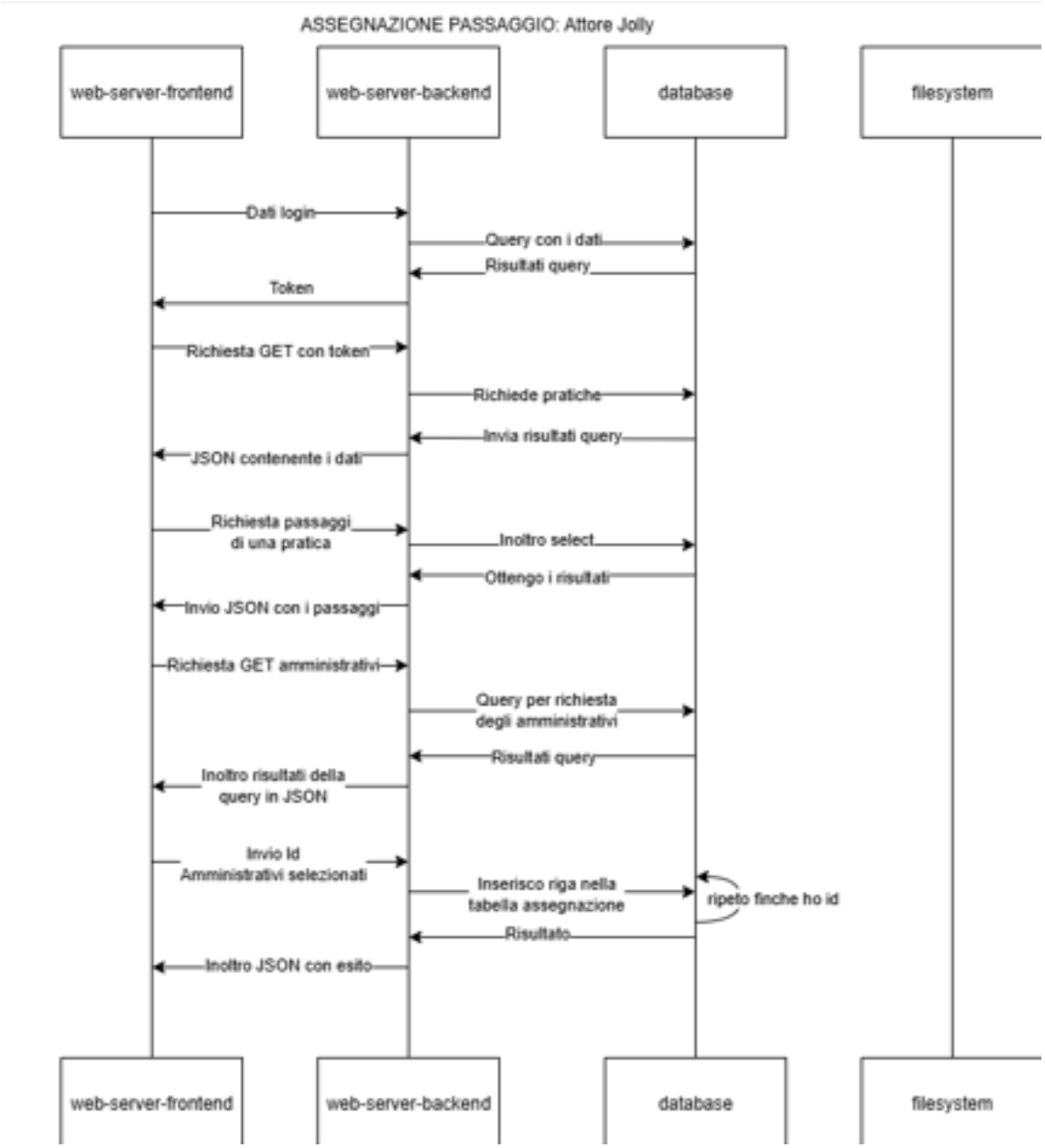

Figure 16: Diagramma dell'assegnamento dei passaggi con Jolly

6.4.6 Eliminazione della pratica

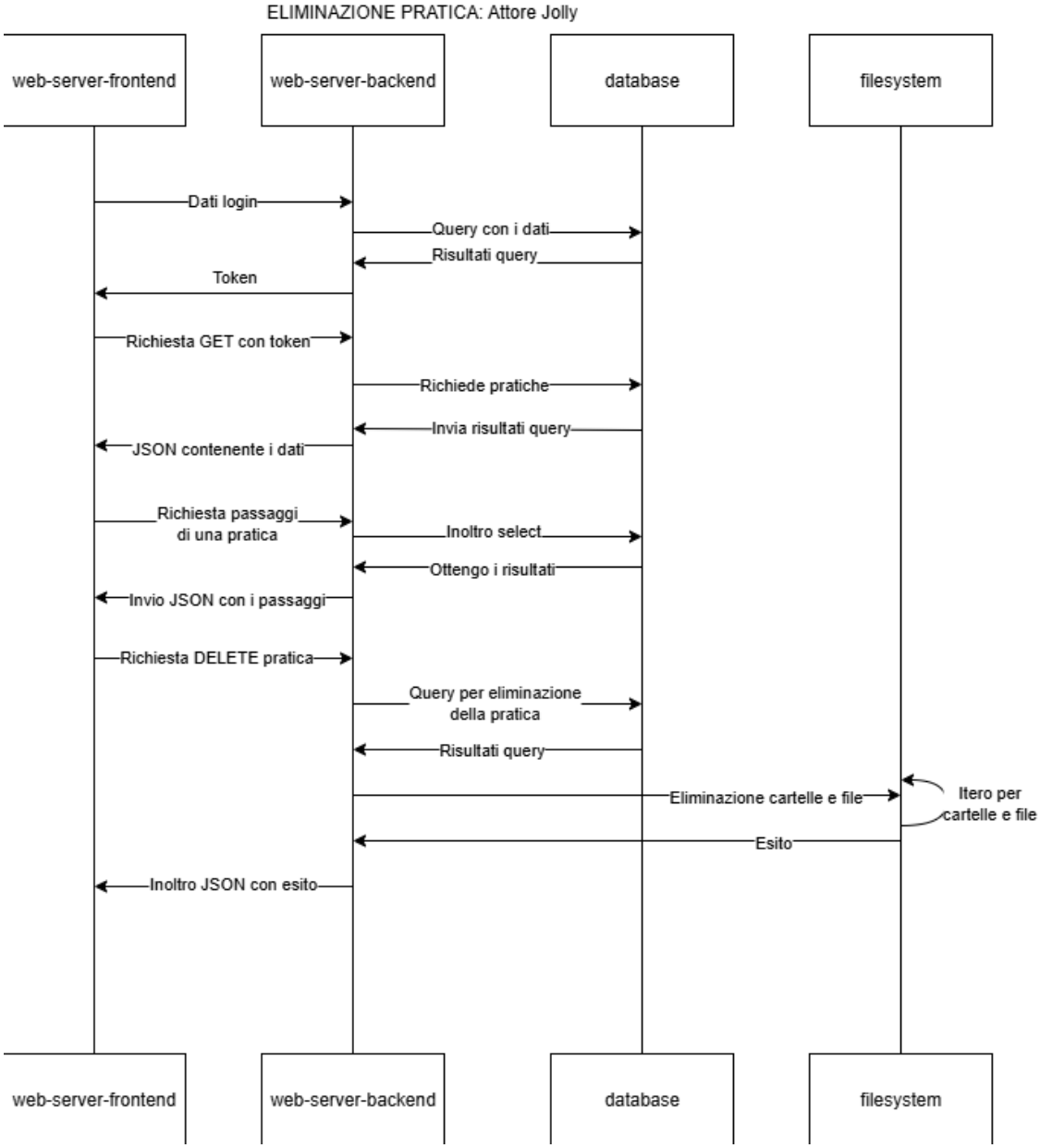

Figure 17: Diagramma dell'eliminazione delle pratiche

6.4.7 Terminazione forzata delle pratiche

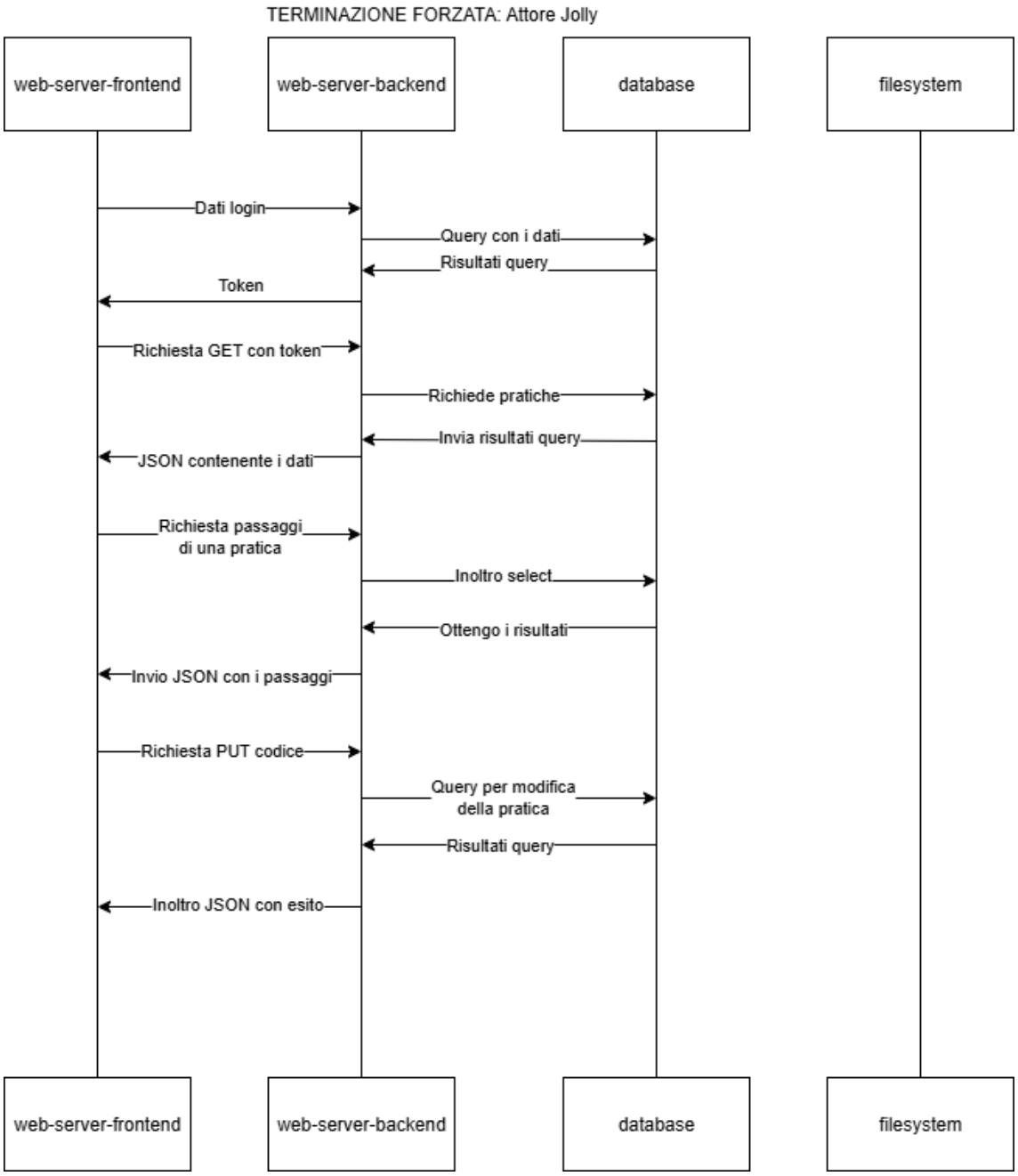

Figure 18: Diagramma della terminazione forzata

6.5 Diagrammi delle attività

6.5.1 Login

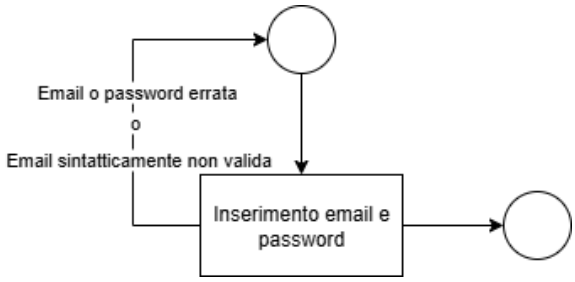

Figure 19: Diagramma delle attività del login

6.5.2 Visualizzazione passaggio

Figure 20: Diagramma delle attività della visualizzazione dei dati di un passaggio

6.5.3 Apertura di una pratica

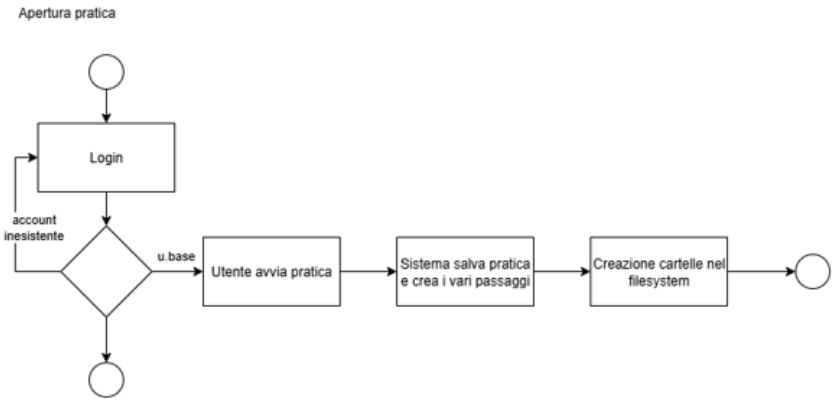

Figure 21: Diagramma delle attività dell'apertura di una pratica

6.5.4 Assegnazione di un passaggio

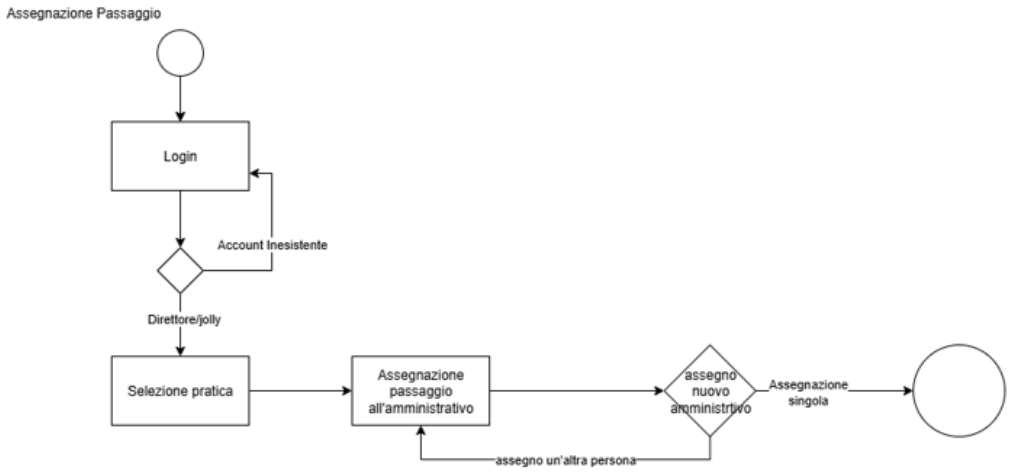

Figure 22: Diagramma delle attività dell'assegnazione di un passaggio

6.5.5 Caricamento dei documenti

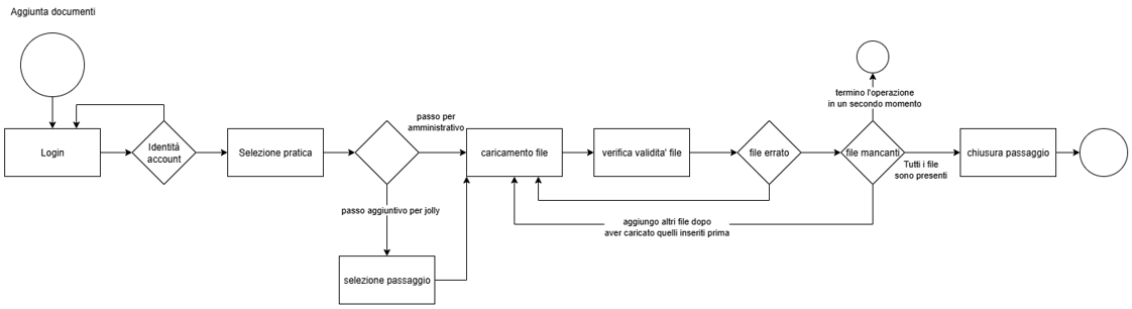

Figure 23: Diagramma delle attività del caricamento di un documento

6.5.6 Terminazione forzata della pratica

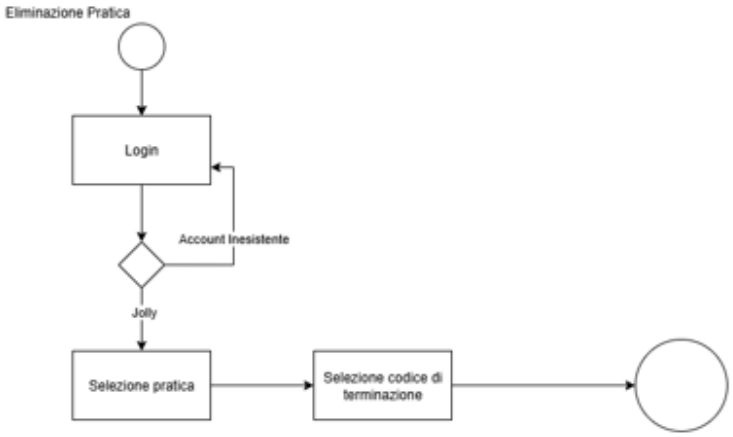

Figure 24: Diagramma delle attività per la terminazione forzata della pratica

6.6 Diagramma delle partizioni

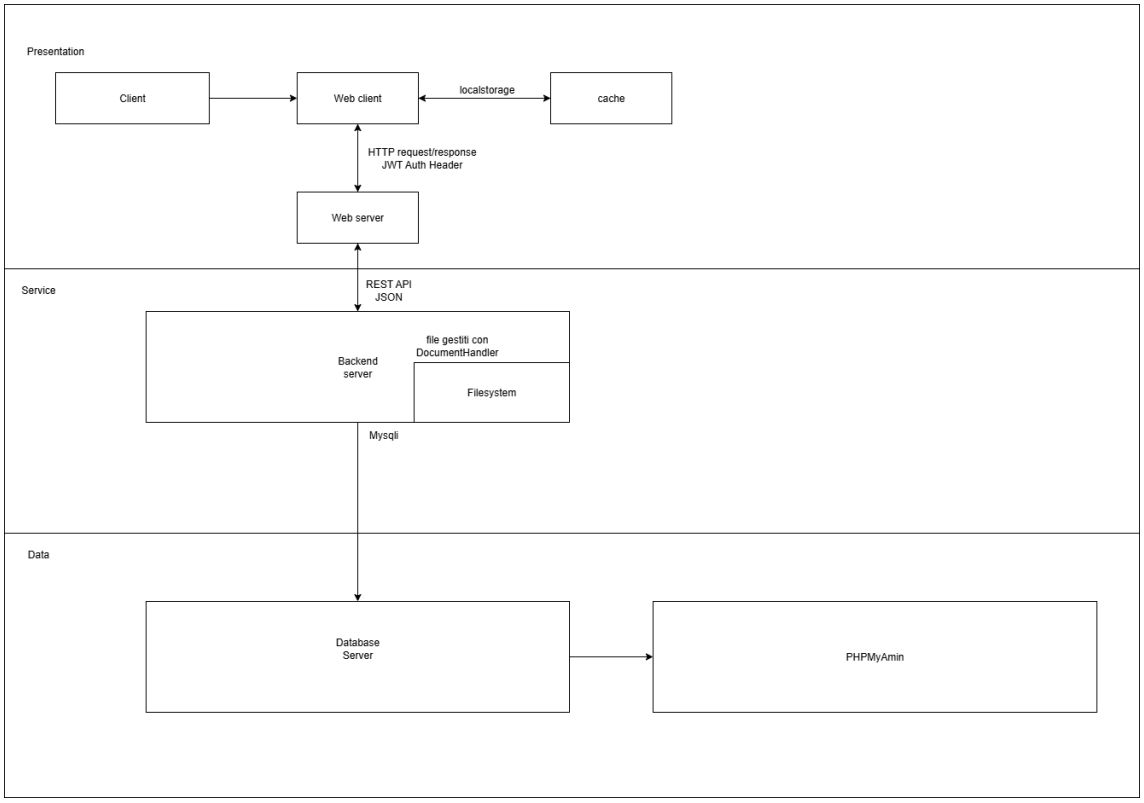

Figure 25: Diagramma delle partizioni

6.7 Diagrammi delle comunicazioni

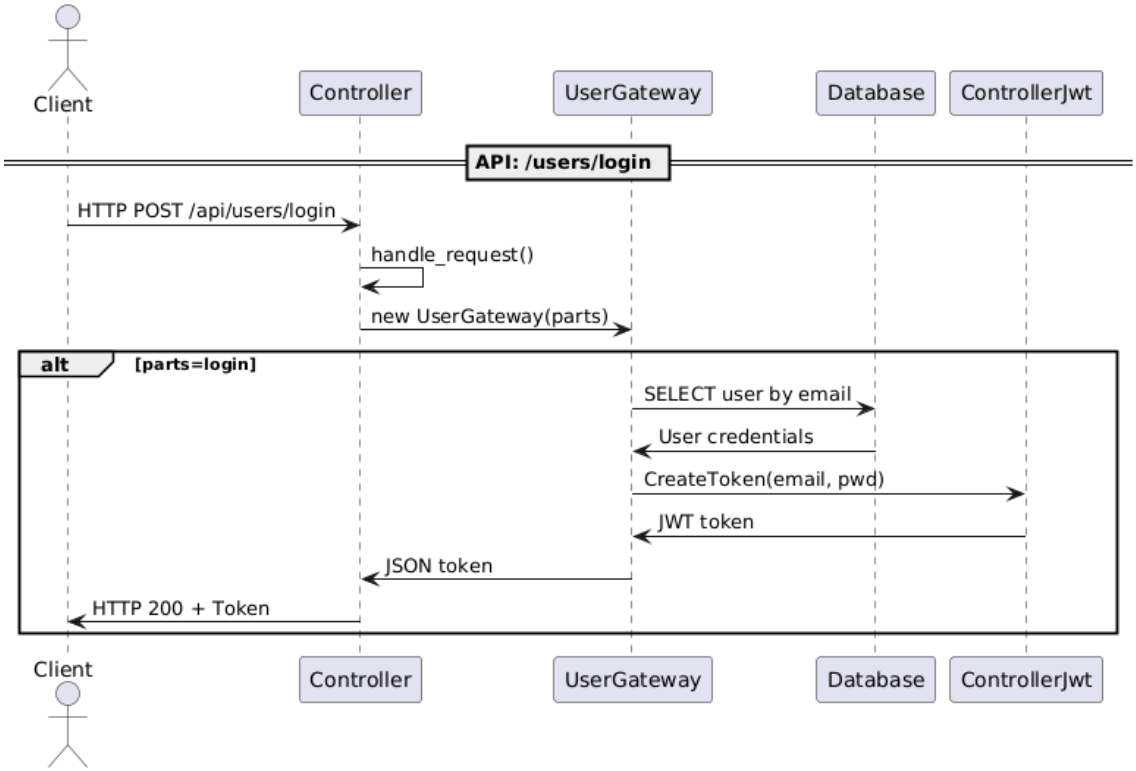

Figure 26: Diagramma di comunicazione dell'API login

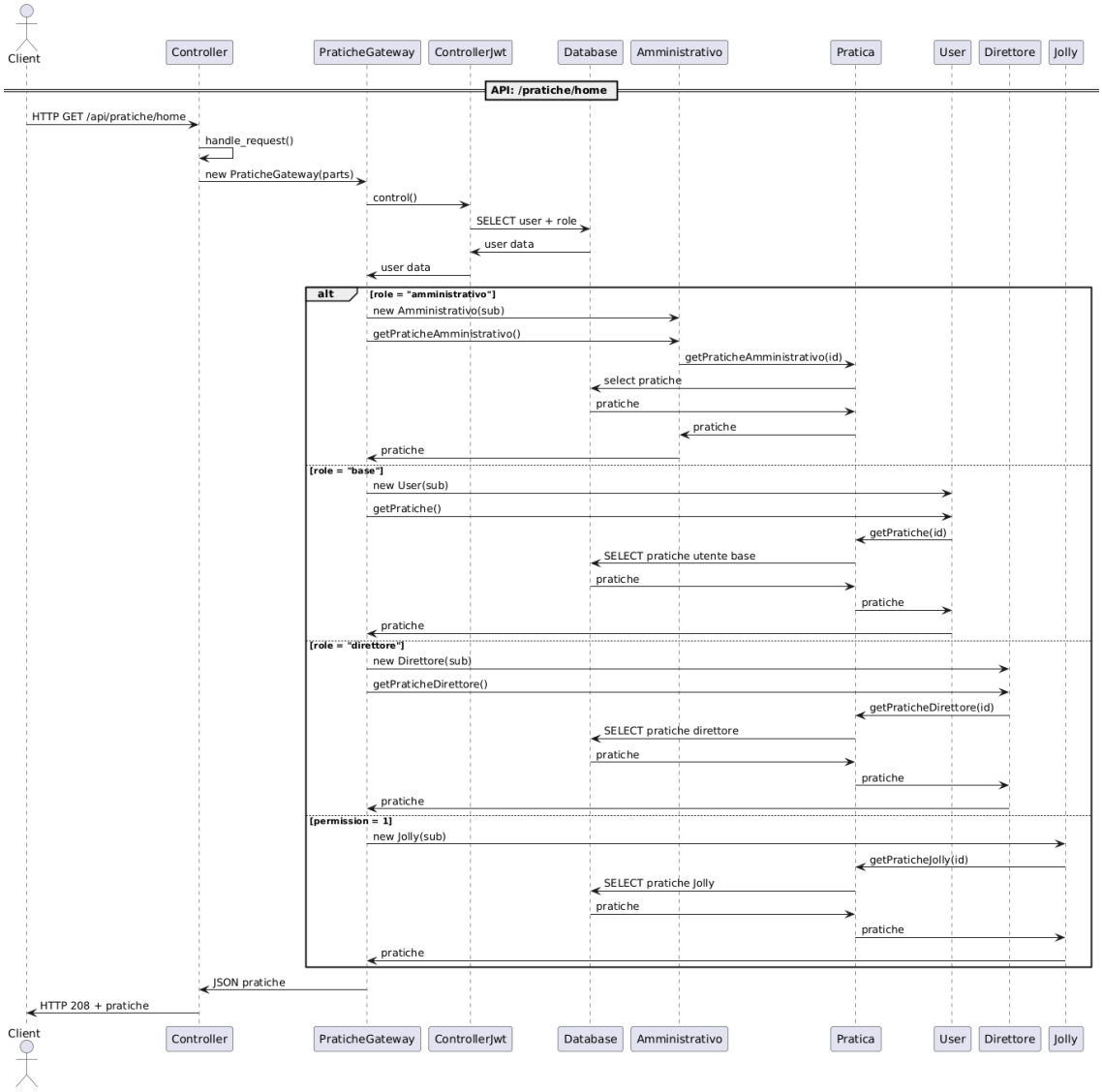

Figure 27: Diagramma di comunicazione dell'API home

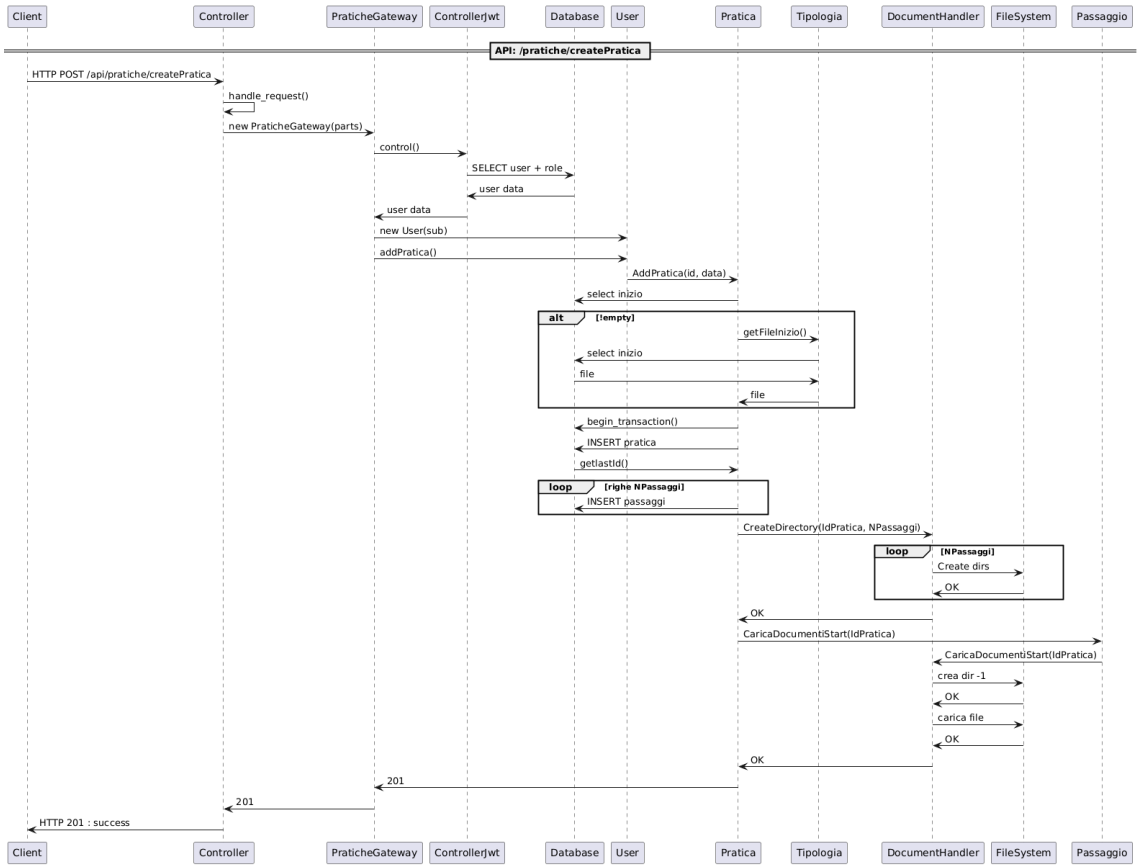

Figure 28: Diagramma di comunicazione dell'API create

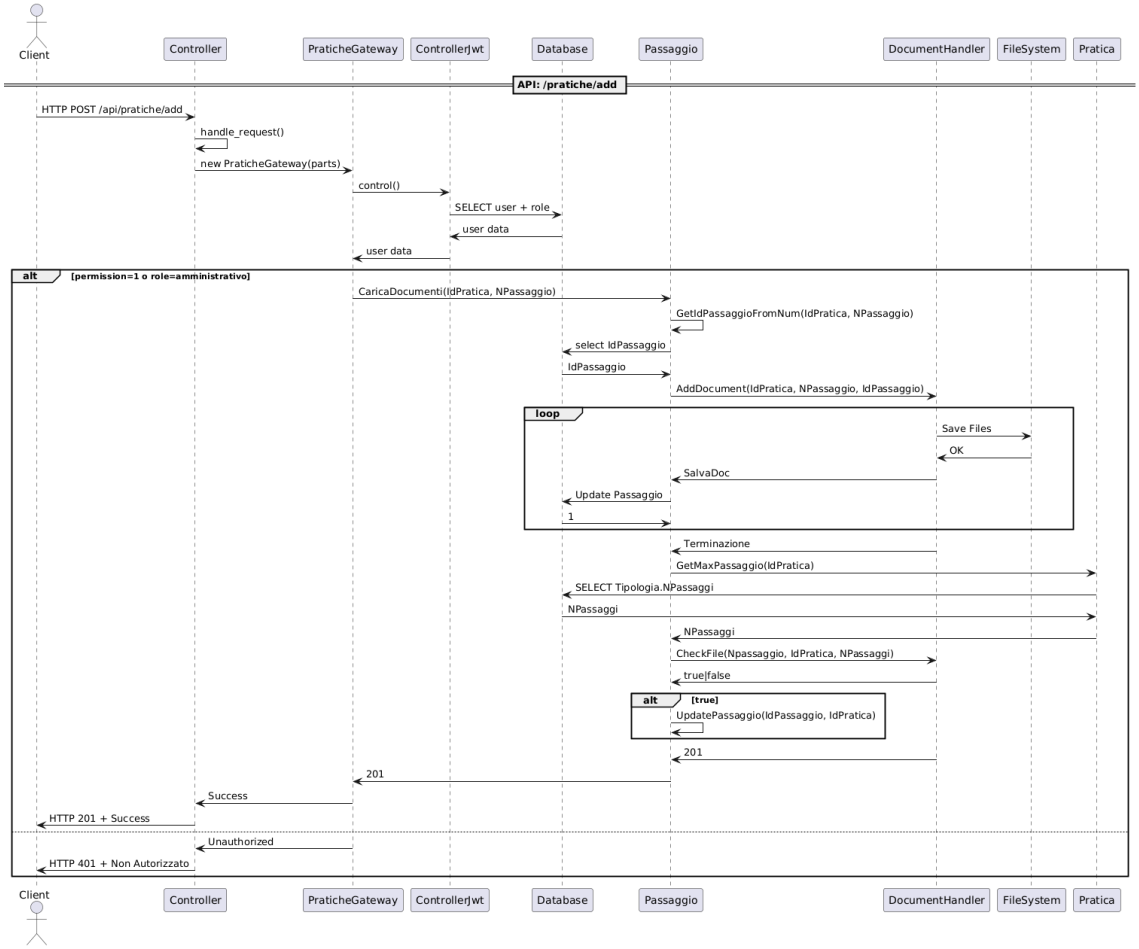

Figure 29: Diagramma di comunicazione dell'API add

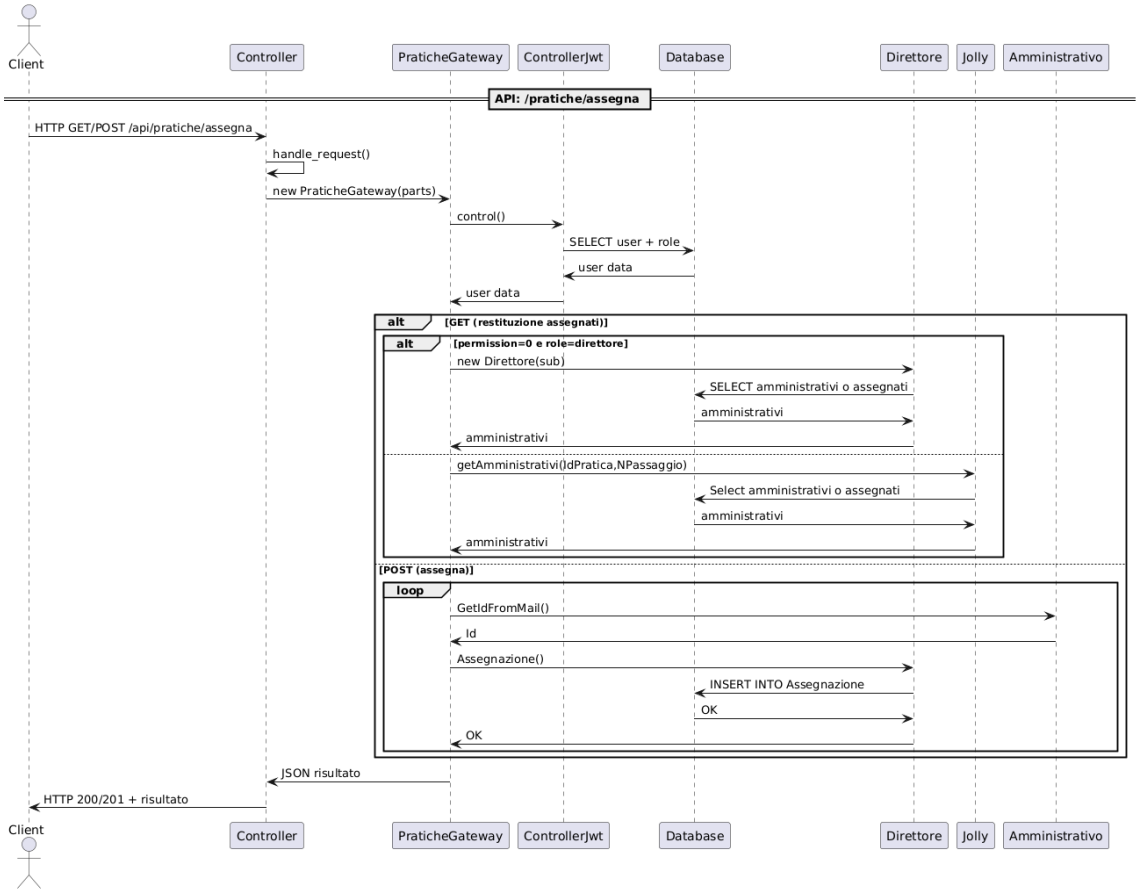

Figure 30: Diagramma di comunicazione dell'API assegna

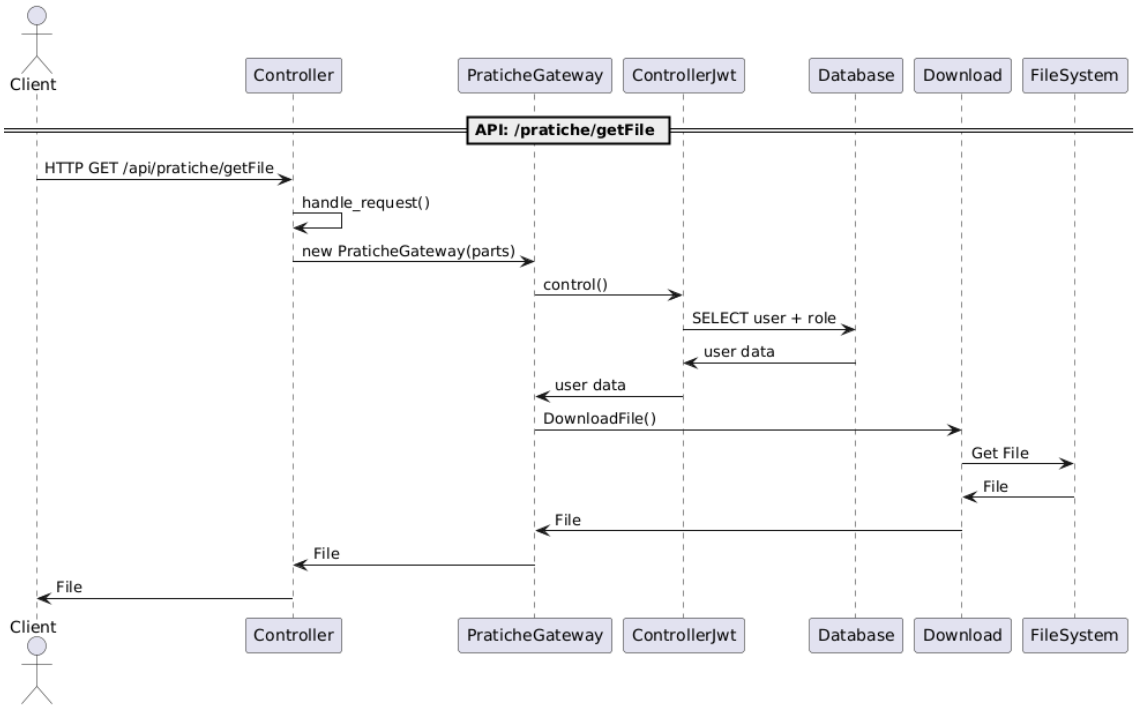

Figure 31: Diagramma di comunicazione dell'API getFile

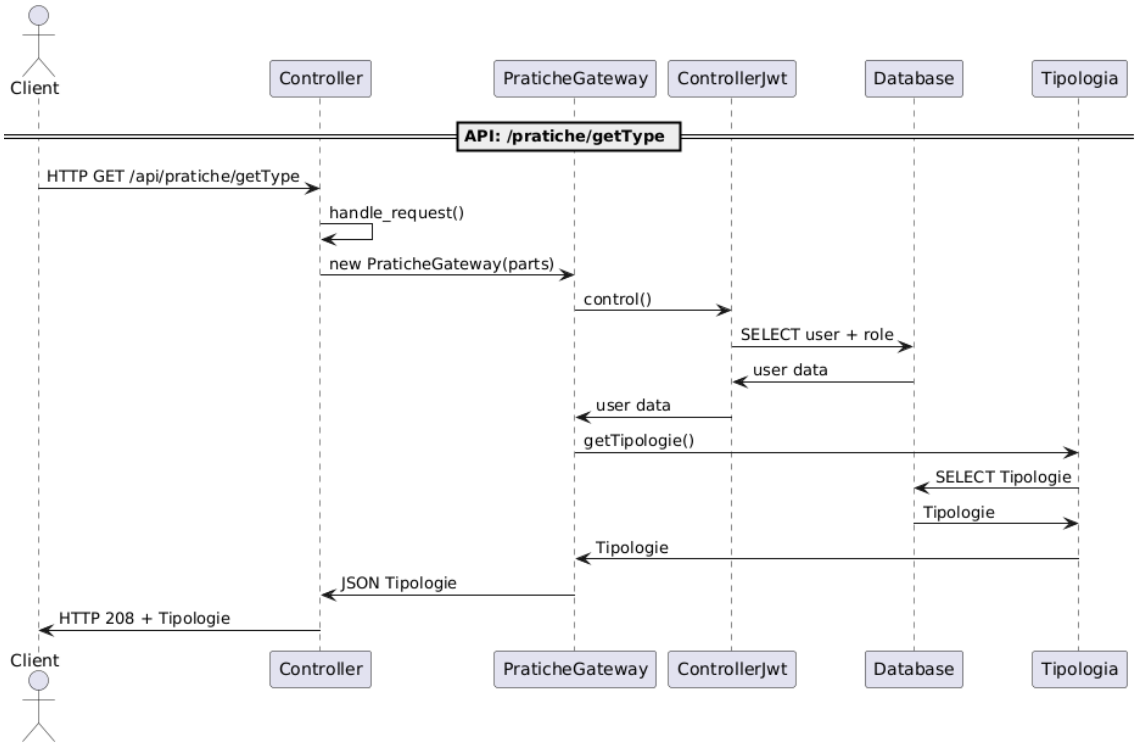

Figure 32: Diagramma di comunicazione dell'API tipologie

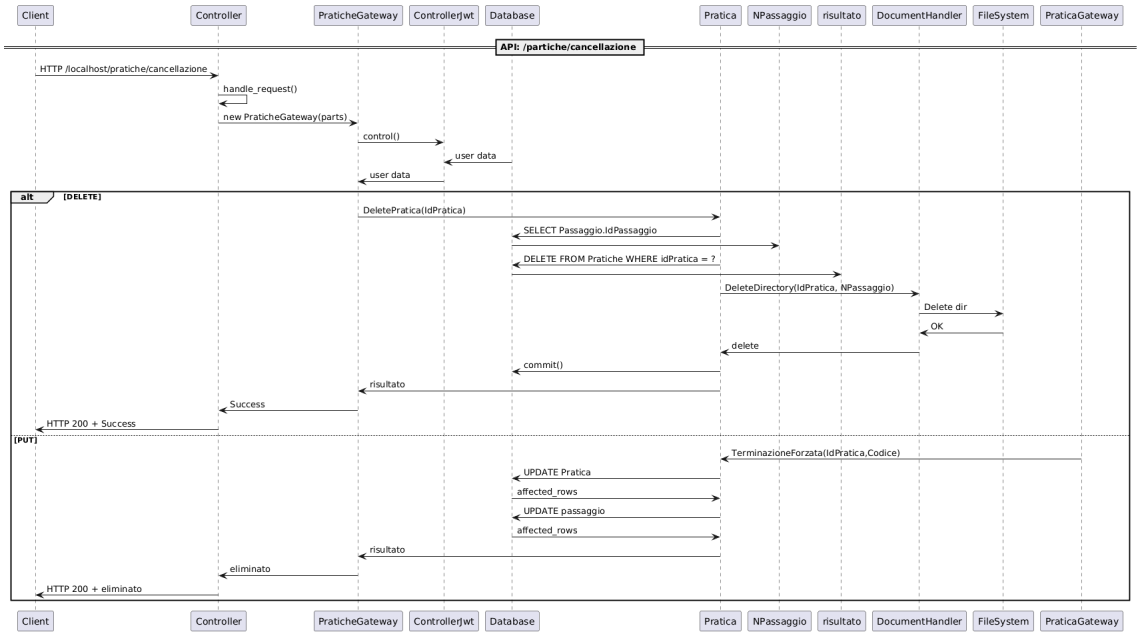

Figure 33: Diagramma di comunicazione dell'API delete

7 Diagramma dei componenti

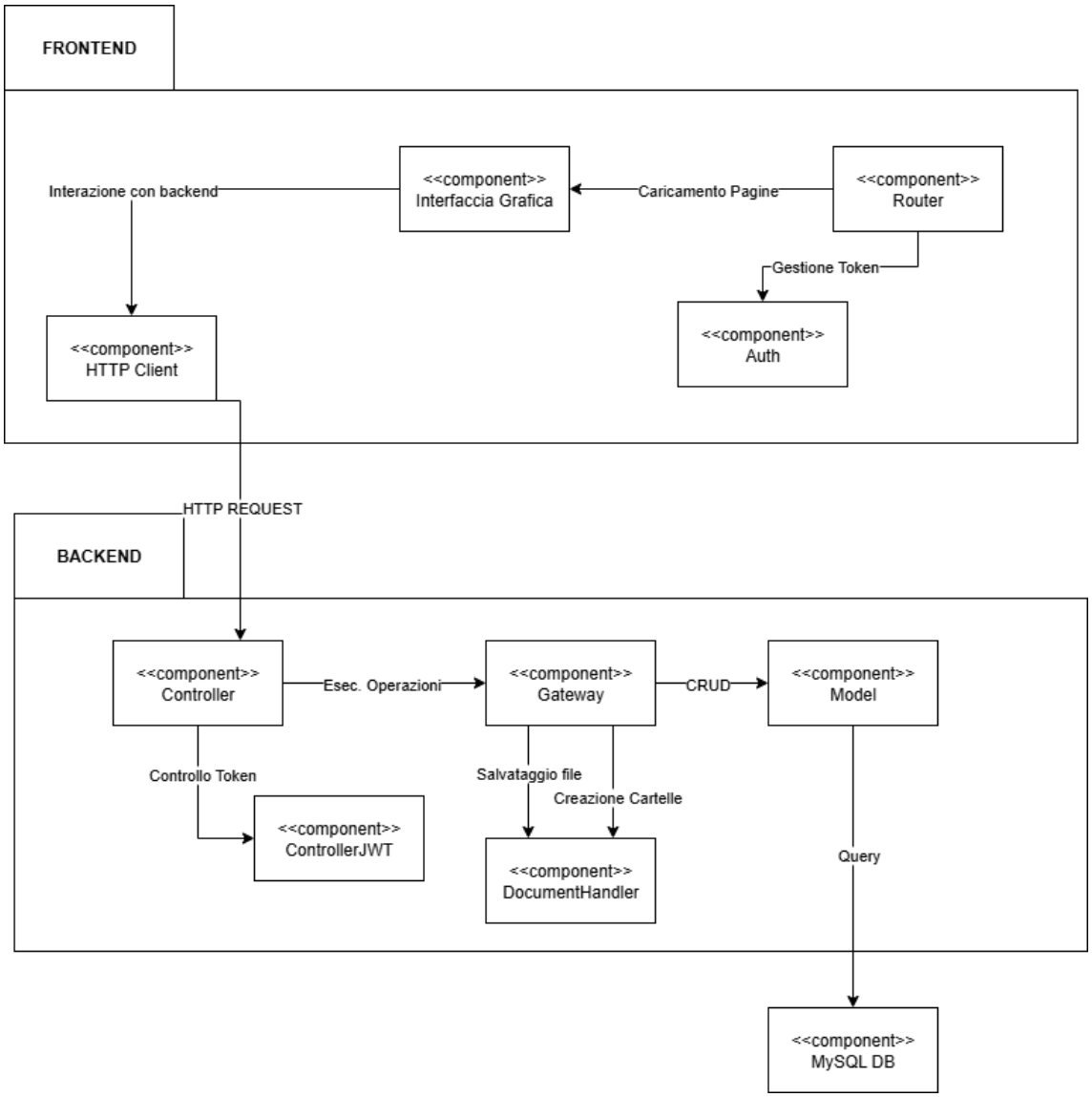

Figure 34: Diagramma dei componenti

8 Diagramma di distribuzione

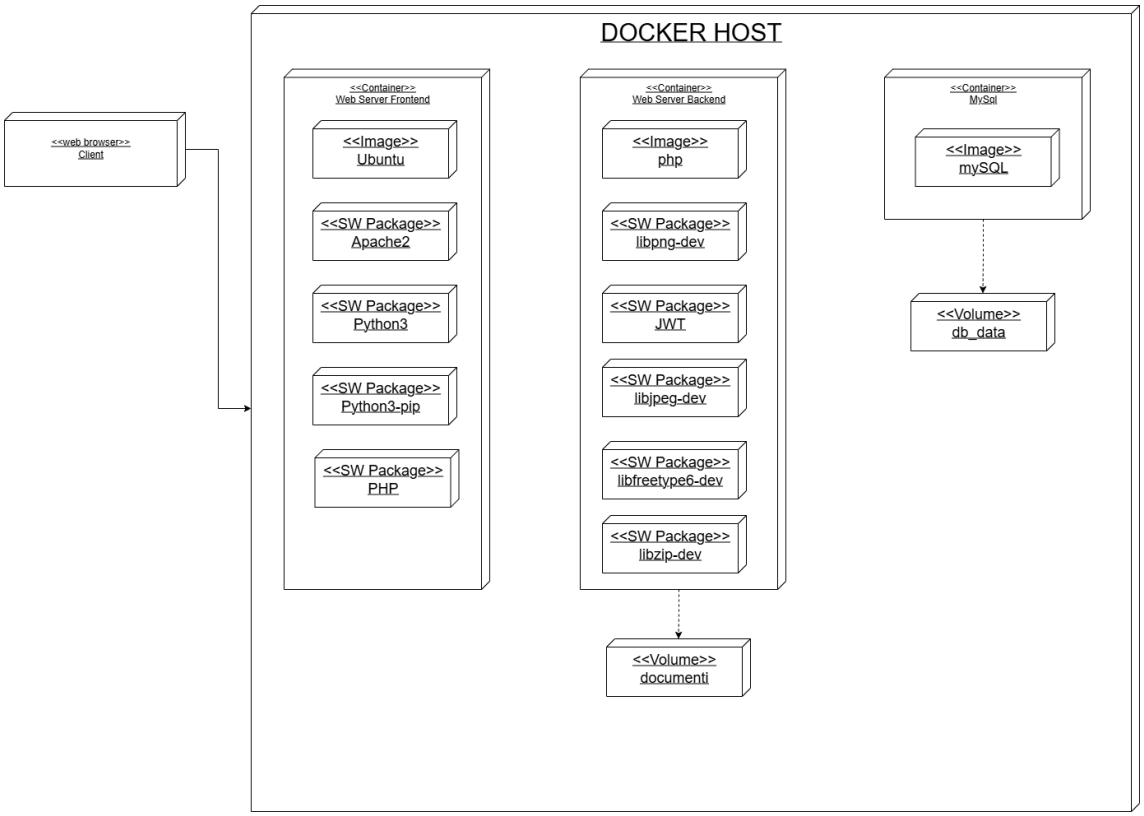

Figure 35: Diagramma di distribuzione

9 Dettagli dell'implementazione

9.1 Frontend

9.1.1 Librerie e strumenti

Per la realizzazione del frontend sono stati utilizzati i seguenti strumenti:

- HTML5 e CSS3: usati per la struttura di base e per il design delle pagine;
- JavaScript: utilizzato per mostrare i contenuti dinamici nelle pagine;
- JQuery: utilizzato per la comunicazione con il backend tramite le chiamate AJAX;
- Bootstrap 5.3: utilizzato per velocizzare la realizzazione dell'interfaccia tramite contenuti già pronti.

9.1.2 Struttura file e cartelle frontend

```
frontend\
|-- assets\
|     |-- img\
|           |-- sfondoLogin.jpg
|-- css\
|     |-- admin.css
|     |-- chief.css
|     |-- jolly.css
```

```
|     |-- navbar.css
|     |-- nuovaPratica.css
|     |-- panel.css
|-- js\
|     |-- loader.js
|     |-- navbar.js
|     |-- pratichеAdmin.js
|     |-- pratichеChief.js
|     |-- pratichеJolly.js
|     |-- pratichеUser.js
|     |-- token.js
|     |-- utils.js
|-- user\
|     |-- nuovaPratica.html
|-- .htaccess
|-- admin.html
|-- chief.html
|-- index.html
|-- jolly.html
|-- user.html
|-- favicon.ico
```

In particolare:

- .htaccess ha il compito di reindirizzare tutte le richieste di pagine non esistenti alla pagina di login (index.html) e di impedire l'accesso a tutti i file che iniziano con un punto;
- il file index.html è il file di login e ad esso vengono reindirizzate tutte le richieste di utenti non autenticati o fatte a pagine non esistenti;
- user.html, admin.html, chief.html e jolly.html sono le pagine relative rispettivamente a docenti/ricercatori, amministrativi, direttori e jolly;
- nuovaPratica.html è la pagina in cui si creano nuove pratiche
- token.js contiene tutte le funzioni JS relative alla gestione dei token da parte del frontend;
- utils.js contiene funzioni utilizzate in tutte le pagine dell'applicazione;
- loader.js contiene le funzioni associate alla creazione delle pratiche;
- la cartella assets conterrà tutti i file di supporto, come ad esempio le immagini;

9.2 Backend

9.2.1 Librerie e strumenti

Le librerie e gli strumenti forniti per l'utilizzo e la realizzazione delle principali funzionalità del backend sono:

- **PHP**: Utilizzato per realizzare la logica del backend, utilizzando il patern MVP.
- **MySQL**: utilizzato per il salvataggio dati sul server.
- **JWT**: utilizzato per la realizzazione di token utilizzabili per le sessioni utente e l'autentiazione prima di effettuare le operazioni CRUD.

9.2.2 Struttura file e cartelle backend

```
backend/
|-- documenti/
|-- Imports/
|   |-- Class/
|   |   |-- Amministrativo.php
|   |   |-- Direttore.php
|   |   |-- Jolly.php
|   |   |-- Passaggio.php
|   |   |-- Pratica.php
|   |   |-- Tipologia.php
|   |   -- User.php
|   -- Interfaces/
|       |-- AmministrativoInterface.php
|       |-- DirettoreInterface.php
|       |-- DocInterface.php
|       |-- DownloadInterface.php
|       |-- JollyInterface.php
|       |-- PassaggioInterface.php
|       |-- PraticaInterface.php
|       |-- TipologiaInterface.php
|       -- UserInterface.php
|-- ControllerJwt.php
|-- Database.php
|-- DocumentHandler.php
|-- Download.php
|-- .htaccess
|-- Controller.php
|-- Gateway.php
|-- index.php
|-- PraticheGateway.php
|-- UserGateway.php
```

- **.htaccess**
  Mi imposta index.php alla prima pagina raggiungibile dalle chiamate al server.

- **Index.php**
  Punto d'accesso al backend, smista le richieste al Controller.

- **Controller.php**
  Opera come punto per lo smistamento delle richieste verso i gateway corretti, quali UserGatewat e PraticheGateway.

- **Gateway.php**
  Classe astratta usata come base per la generazione delle singole classi di gateway che implementano il metodo handle_request. In particolare il gateway Usergateway.php è impostato per la realizzazione del token da restituire per le richieste di login; mentre PraticheGateway.php si occupa della gestione delle operazioni quali creazione, assegnazione, eliminazione dei passaggi e delle pratiche e il caricamento dei file.

- **Database**
  Contiene le funzioni per inviare richieste al database e gestire le transaction.

- **ControllerJWT**
  La classe serve per realizzare e controllare il token d'accesso al servizio.

- **DocumentHandler**
  La classe serve per effettuare tutte le singole operazioni sul filesystem e il check dei file

caricati tramite la tabella del database per richiamare, in caso di necessita', la funzione di terminazione del passaggio/pratica.

- **Download**
  Classe che permette, tramite una richiesta get fatta dall'amministrativo o jolly, di scaricare i file caricati per verificare che i file inseriti siano corretti.

9.2.3 Autenticazione

L'autenticazione viene effettuata tramite il controllo del JWT salvato nell'header della richiesta JSON. Contiene informazioni riguardanti il tempo di creazione del token, la tipologia dell'utente, un valore numerico (1 o 0) per indicare se l'utente è jolly, il nome dell'utente, il suo id nel database e infine il tempo di scadenza del token (posto a 1 ora). Per eseguire qualsiasi operazione all'interno del servizio l'utente avrà bisogno prima di tutto di verificare la sua identità; una volta fatto ciò il token sarà utilizzato per verificare l'utenza e anche verificare che il token non sia stato alterato durante l'invio o nel frontend.

9.2.4 Caricamento documenti

Alla creazione della pratica viene effettuata una transaction la quale termina solamente se a livello di database siano stati creati anche i vari passaggi della pratica e se a livello di filesystem siano state create le cartelle della pratica e dei passaggi; se questi due eventi sono terminati con successo allora esegue il commit della transaction, altrimenti il rollback. Si previene anche il caso in cui i file caricati non siano corretti o siano già stati inseriti.

9.3 COMUNICAZIONE API

```
POST /users/login
Body
{
    email: 'string'
    password: 'string'
}
```

JSON per login

```
POST /pratiche/home
Headers
{
    Authorization: Bearer <token>
}
```

JSON per la dashboard

```
GET /pratiche/getType
Headers
{
    Authorization: Bearer <token>
}
```

JSON per ottenere le tipologie e i documenti iniziali richiesti

```
POST /pratiche/createPratica
Headers
{
```

```
    Authorization: Bearer <token>
}
Data
{
    Tipologia: ‘string’
    Descrizione: ‘string’
    files[]: [<formato>]
}
```

JSON per ottenere la creazione della pratica

```
GET /pratiche/assegna
Headers
{
    Authorization: Bearer <token>
}
```

JSON per ottenere gli amministrativi dell’Unita’ Operativa

```
POST /pratiche/assegna
Headers
{
    Authorization: Bearer <token>
}
Data
{
    IdPratica: ‘int’
    NPassaggio: ‘int’
    Mail[]: [string]
}
```

JSON per assegnare gli amministrativi dell’Unita’ Operativa indicati

```
DELETE /pratiche/cancellazione
Headers
{
    Authorization: Bearer <token>
}
Data
{
    IdPratica: ‘int’
}
```

JSON per eliminare la pratica

```
DELETE /pratiche/cancellazione
Headers
{
    Authorization: Bearer <token>
}
Data
{
    IdPratica: ‘int’
    Codice: ‘int’
}
```

JSON per la terminazione forzata di una pratica

10 Testing

Il testing è stato effettuato al termine e durante ogni attività di produzione di codice delle operazioni CRUD. In particolare sono stati effettuati test al termine della produzione del backend per la parte del login tramite l'uso di **postman**.
Successivamente è stata fatta l'integrazione tra frontend e backend del solo login e in seguito **testata manualmente**. Continuando è stato prodotto il codice per l'utente base sia a livello di frontend che a livello di backend, dopodiché è stato testato prima il backend, sempre tramite **postman**, in seguito è stata fatta l'unione delle due parti ed è stata anch'essa testata; così è stato fatto anche per la parte relativa alle operazioni dell'amministrativo, del direttore e infine del jolly.
È anche stato tentato il testing automatico tramite PHPunit, il quale è stato utile per verificare la corretta restituzione delle pratiche e dei passaggi; ha però causato problemi nella creazione delle cartelle, durante il testing della creazione delle pratiche; poiché PHPUnit utilizza l'utente root per la creazione delle cartelle, mentre il codice php del backend utilizza lo user `www-data`. Il caricamento dei file, così come la terminazione di un passaggio e della pratica, sono stati verificati tramite l'inserimento manuale di file; si è verificato tramite quell'operazione anche la chiusura automatica di più passaggi purché siano presenti i file indicati nella lista dei documenti d'uscita.

10.1 PHPUnit

```
$user = new User(7);
$this->assertEquals(7,$user->getId());
...
```

Verifico che lo user selezionato sia corretto. Creo la pratica manualmente per evitare limiti riguardanti i file.

```
$pratica = $user->getPratiche();
$this->assertEquals(1,$pratica[0]["IdPratica"]);
...
```

Verifico successivamente che la pratica che ricevo sia quella creata in precedenza.

```
$direttore = new Direttore(1);
$this->assertEquals(1,$direttore->getDirettore());
$pratica=$direttore->getPraticheDirettore();
...
Direttore::Assegnazione(1,3);

$direttore->method('getAmministrativi')
              ->willReturn([
                  'successo' => 'Assegnazione avvenuta con successo',
                  'amministrativi' => [
                      ['Mail' => 'anna.verdi@example.com'],
                      ['Mail' => 'giulia.neri@example.com'],
                      ['Mail' => 'maria.rossa@example.com']
              ]
]);
$result = $direttore->getAmministrativi();
```

Verifico con il direttore del passaggio 1 della tipologia A che sia presente il passaggio della pratica e verifico l'assegnazione.
Successivamente verifico gli stessi passaggi del direttore con il Jolly.

11 Avvio

Per avviare il progetto è sufficiente avviare docker desktop ed eseguire il comando
docker-compose up -d
di suo scaricherà le immagini richieste con anche i moduli necessari.
All'avvio nel database saranno già presenti degli utenti con i quali si potrà accedere ai servizi; questo è stato fatto, senza pagina di creazione dell'utente, per simulare una finta connessione a database contenenti già gli utenti; poiché si è pensato che, essendo un servizio universitario, questo sarà connesso a server dell'università.